UMLytics

AI-Powered UML Generation and Design Intelligence Tool

Software Requirements and Design Document

Ammar Bin Omer — 24I-0500
Haris Zahid — 24I-0643
Shahmeer Jadoon — 24I-0879

May 2026

Department of Computer Science
National University of Computer and Emerging Sciences
Islamabad Campus

Contents

1 Introduction

1.1 Purpose

This document is the Software Requirements and Design Specification (SRS/SDD) for UMLytics, an AI-powered desktop application for generating, editing, and evaluating UML class diagrams. It defines the complete scope, requirements, architecture, and design decisions for the system. The document serves as the authoritative reference for the development team, academic reviewers, and any future maintainers of the UMLytics codebase. It covers all twelve use cases (UC1-UC12), the domain model, system and design sequence diagrams, and the layered class architecture implemented using GRASP principles and GoF patterns.

1.2 Product Scope

UMLytics is a Java desktop application that addresses the gap between natural-language software descriptions and formal UML class diagrams. A software designer can describe their intended system in plain English, upload existing Java source code, or sketch a diagram manually—and the system will generate, refine, and evaluate a UML class diagram with AI assistance.

The scope includes:

- AI-driven UML class diagram generation from natural language prompts and Java source code.
- A draw.io-inspired interactive canvas for manual diagram creation and editing.
- Design quality evaluation using SOLID principles, cohesion, and coupling metrics.
- An AI co-pilot chat panel for real-time design guidance.
- Voice-based input for hands-free diagram creation.
- Diagram export in PNG, SVG, and PDF formats.
- Full project persistence using a local SQLite database.

1.3 Title

UMLytics — AI-Powered UML Generation and Design Intelligence Tool

1.4 Objectives

- Reduce the time required to produce a first-draft UML class diagram from hours to under two minutes using LLM-backed generation.
- Provide instant, actionable design feedback covering SOLID principle adherence, coupling score, and cohesion score without requiring an external tool.
- Enable code-first reverse engineering: accept Java source files and automatically reconstruct the corresponding UML class diagram via AST traversal.
- Deliver a pixel-faithful draw.io canvas experience within a standalone JavaFX application—no browser dependency.
- Support voice input as a first-class input method alongside keyboard and mouse, broadening accessibility.
- Ensure all project data (diagrams, chat history, evaluations) persists locally in SQLite, enabling offline-first operation.

1.5 Problem Statement

Creating UML diagrams is an indispensable part of software design, yet it remains one of the most time-consuming and error-prone phases of the development lifecycle. Existing diagramming tools such as draw.io, Lucidchart, and Visual Paradigm require the designer to manually place each class box, type every attribute and method, and draw every relationship edge. For a moderately complex system with twenty or more classes this process can consume hours that would be better spent on actual architectural reasoning.

Furthermore, even after a diagram is produced, there is no automated mechanism in these tools to evaluate whether the design satisfies fundamental quality principles such as SOLID, low coupling, and high cohesion. Designers must rely on code review sessions or hire separate consultants to catch these issues—a process that is both slow and costly in educational and small-team contexts.

UMLytics solves both problems in a single, self-contained application. By integrating a large language model directly into the diagram editing workflow, it transforms a natural-language requirement description into a fully-structured UML class diagram in seconds. Its built-in evaluation engine then scores the resulting design against industry-standard metrics and explains every flagged violation in plain English, closing the feedback loop without leaving the application.

2 Overall Description

2.1 Product Perspective

UMLytics is a standalone desktop application with no dependency on a web server or cloud infrastructure beyond the external LLM API call. It runs as a native JavaFX 21 application on Windows, macOS, and Linux operating systems with JDK 17 or later installed. The system is structured as a four-layer architecture: a JavaFX-based Presentation Layer, a GRASP Controller Layer, a Service and Repository Layer, and a SQLite Data Layer. All inter-layer communication is mediated through Java interfaces, making each layer independently testable and substitutable. The LLM integration uses the Anthropic/OpenAI chat completions API via OkHttp for HTTP communication and Gson for JSON serialisation. The Java source code analysis component uses the JavaParser library to produce Abstract Syntax Trees (ASTs) from which UML class members are extracted. Diagram export uses Apache PDFBox and Apache Batik for PDF rendering and SVG rendering respectively.

2.2 Product Functions

The major functional areas of UMLytics are:

- **Project Management:** Create, open, rename, and delete design workspaces that persist all diagrams, evaluations, and chat history.
- **UML Generation (NL):** Translate a natural-language system description into a UML class diagram via LLM, with full class names, typed attributes, method signatures, and relationship labels.
- **UML Generation (Code):** Reverse-engineer one or more Java source files into a UML diagram using AST analysis, preserving visibility modifiers and generics.
- **Manual Diagram Editing:** Add, remove, rename, and reposition class nodes; draw and delete relationships using a draw.io-style toolbar and canvas.
- **AI Design Co-Pilot:** Ask open-ended design questions in a sidebar chat; the AI has full visibility of the current diagram as context.

- **Design Quality Evaluation:** Automatically score a diagram for coupling, cohesion, and SOLID principle compliance, with a written feedback summary.
- **Class Skeleton Suggestions:** Request Java skeleton code for any class or the entire diagram, generated by the LLM.
- **Diagram Image Analysis:** Upload a hand-drawn or photographed UML diagram image; the Vision AI reconstructs it as an editable digital diagram.
- **Voice Input:** Dictate diagram creation commands via microphone using a speech-to-text service.
- **Export:** Save the current diagram canvas to PNG, SVG, or PDF.

2.3 List of Use Cases

UC #	Use Case Name	Brief Description
UC1	Create / Initialize Project	Software designer creates a new named project workspace.
UC2	Generate Diagram from Natural Language	System generates a UML class diagram from a text description.
UC3	Generate Diagram from Source Code	System parses Java files and produces a reverse-engineered diagram.
UC4	Manually Edit Diagram	Designer adds, removes, or repositions classes and relationships on the canvas.
UC5	Evaluate Design Quality	AI evaluates the diagram for SOLID, coupling, and cohesion.
UC6	Consult AI Design Guidance	Designer asks the AI co-pilot a design question.
UC7	Generate Class Suggestions	AI generates skeleton Java code for selected classes.
UC8	Analyze Uploaded Diagram Image	Vision AI reconstructs a diagram from an uploaded PNG/JPG.
UC9	Maintain Project Data	Designer renames, archives, or deletes a project and its diagrams.
UC10	Retrieve Existing Project	Designer opens and loads a previously saved project.
UC11	Voice-Based Input	Designer dictates a diagram description using a microphone.
UC12	Export UML Diagram	Designer saves the canvas to PNG, SVG, or PDF.

2.4 Extended Use Cases

UC1 — Create / Initialize Project

Name	UC1 — Create / Initialize Project
Scope	Software designer creates a named, empty project workspace.
Primary Actor	Software Designer
Stakeholders	Software Designer, System Administrator
Precondition	Application is running and no existing project has the same name.
Postcondition	A new Project record is persisted in SQLite; the dashboard panel refreshes.
Success Scenario	1. Designer clicks 'New Project'. 2. Enters project name and optional description. 3. System validates uniqueness via ProjectController. 4. ProjectRepositoryImpl inserts the record. 5. Dashboard panel refreshes with the new project tile.
Extension (Dup)	3a. Name already exists → ValidationException is thrown → toast notification 'Project name already in use.'
Special Req.	Project name must be 1–120 characters, no leading whitespace.

UC2 — Generate Diagram from Natural Language

Name	UC2 — Generate Diagram from Natural Language
Scope	System generates a full UML class diagram from a plain-English description.
Primary Actor	Software Designer
Precondition	A project is open. Description is at least 10 characters.
Postcondition	A UMLDiagram is saved; the diagram canvas renders the generated classes.
Success Scenario	1. Designer types a description (¿10 chars). 2. DiagramController.generateFromText() validates length. 3. LLMAPIEngine sends an OkHttp POST to the LLM API. 4. Response JSON is parsed into a UMLModel. 5. UMLModel.toUMLDiagram() produces a UMLDiagram. 6. DiagramRepositoryImpl persists it. 7. DiagramEditorPanel renders class nodes and edges.
Extension (Short)	2a. Description $\leq$ 10 chars → ValidationException → toast 'Description too short.'
Extension (API)	3a. LLM API unreachable → AIEngineException → toast 'AI service unavailable.'
Special Req.	LLM response must include class names, typed attributes, method signatures, and relationship annotations.

UC3 — Generate Diagram from Source Code

Name	UC3 — Generate Diagram from Source Code
Scope	System parses uploaded Java files and constructs a reverse-engineered class diagram.
Primary Actor	Software Designer
Precondition	Designer provides at least one .java file.
Postcondition	UMLDiagram with SourceType.SOURCE_CODE is persisted and rendered.
Success Scenario	1. Designer selects one or more .java files. 2. DiagramController validates all files end in '.java'. 3. JavaCodeParser uses JavaParser AST traversal to extract ClassOrInterfaceDeclaration nodes. 4. Each declaration maps to a ConceptualClass with its Attributes and Methods. 5. Diagram is persisted and rendered.
Extension (NonJava)	2a. Non-.java file detected → UnsupportedFileException.

UC5 — Evaluate Design Quality

Name	UC5 — Evaluate Design Quality
Scope	AI evaluates the current diagram and produces a DesignEvaluationReport.
Primary Actor	Software Designer
Precondition	An active diagram with at least 2 classes is loaded.
Postcondition	Report with coupling score, cohesion score, SOLID score, and feedback is saved.
Success Scenario	1. Designer clicks 'Evaluate Design'. 2. AIController.evaluateDesign() retrieves diagram. 3. Checks class count ≥ 2 (else DiagramTooSimpleException). 4. UMLModel is serialised and sent to LLMAPIEngine. 5. Response is parsed into DesignEvaluationReport. 6. EvaluationPanel renders scores and feedback text.
Extension (Simple)	3a. ¡ 2 classes → DiagramTooSimpleException → 'Diagram too simple to evaluate.'

UC8 — Analyze Uploaded Diagram Image

Name	UC8 — Analyze Uploaded Diagram Image
Scope	Vision AI reconstructs a hand-drawn or photo-captured UML image into a digital diagram.
Primary Actor	Software Designer
Precondition	Designer has a PNG or JPG image of a UML diagram.
Postcondition	A new UMLDiagram (SourceType.UPLOADED_IMAGE) is persisted and rendered.
Success Scenario	1. Designer uploads image. 2. DiagramController.analyzeUploadedImage() validates magic bytes (PNG/ JPG). 3. Image is base64-encoded and sent to Vision LLM. 4. LLM returns class structure JSON. 5. Diagram is created, saved, and rendered.
Extension (Invalid)	2a. Magic bytes invalid → UnsupportedFileException → 'Only PNG/JPG files are supported.'

UC12 — Export UML Diagram

Name	UC12 — Export UML Diagram
Scope	Designer exports the diagram canvas to a binary image or document file.
Primary Actor	Software Designer
Precondition	At least one class node exists on the canvas. A valid file path is chosen.
Postcondition	A PNG, SVG, or PDF file is written to the chosen path.
Success Scenario	1. Designer selects Export and chooses format (PNG/SVG/PDF). 2. DiagramController.exportDiagram() validates format and path. 3. DiagramExportService uses layout algorithms to compute bounds. 4. Apache PDFBox (PDF) or Batik (SVG) renders the output. 5. File is written; toast confirms success.
Extension (NullFmt)	2a. Format is null → UnsupportedFormatException.
Extension (BlankPath)	2b. Path is blank → ValidationException.

2.5 Use Case Diagram

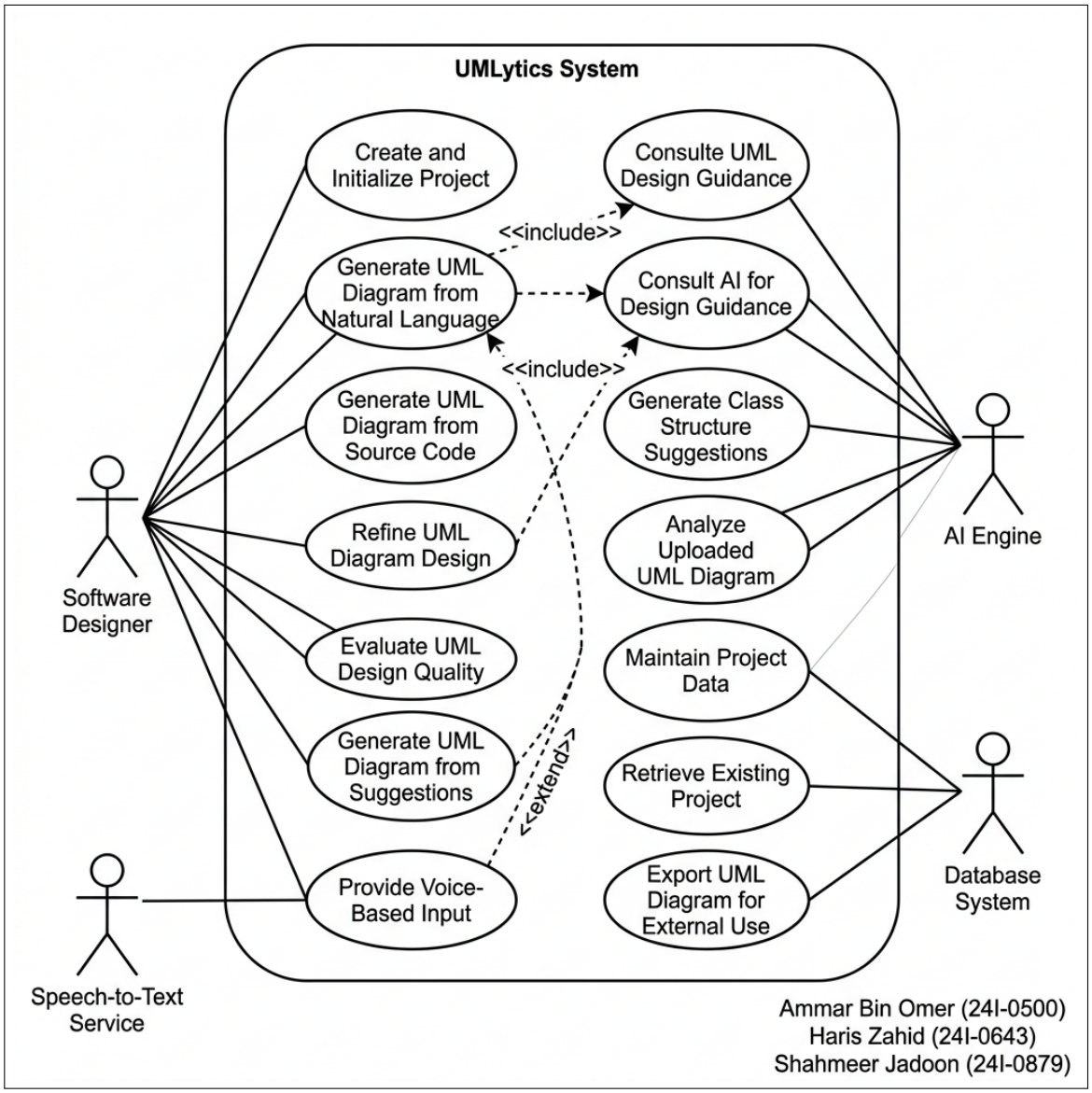

Figure 1: Use Case Diagram

3 Other Non-Functional Requirements

3.1 Performance Requirements

- LLM-based diagram generation must complete within 15 seconds under normal network conditions using the Claude Sonnet or GPT-4o API tier.
- Java AST parsing for a file set of up to 50 source files must complete within 3 seconds on a standard development machine (Intel i5, 8 GB RAM).
- Diagram export to PNG and SVG must complete in under 2 seconds for diagrams containing up to 30 class nodes.
- SQLite database operations (save/load project, fetch diagrams) must not block the JavaFX Application Thread; all JDBC calls are executed on a separate thread via ConnectionPool.

- The UI must remain interactive (60 fps) during background AI processing; progress indicators must appear within 200 ms of initiating any AI operation.

3.2 Safety Requirements

- All AI-generated Java skeleton code must be displayed as non-executable text; the system must never auto-compile or auto-run generated code.
- The application must never modify the source files provided for code analysis—files are opened read-only.
- If a SQLite write operation fails mid-transaction, the transaction must be rolled back to prevent partial/corrupt diagram state.

3.3 Security Requirements

- The LLM API key is stored exclusively in `config.properties` on the local filesystem and is never logged, transmitted to a third party, or included in export files.
- All SQL queries use PreparedStatements with positional parameters to eliminate SQL injection vulnerabilities.
- Magic-byte validation in `analyzeUploadedImage()` prevents execution of non-image payloads as data.
- The application does not transmit any project diagram data to external services beyond the LLM API call, which the user explicitly triggers.

3.4 Software Quality Attributes

- **Maintainability:** Every layer communicates through interfaces (`IAIEngine`, `IDiagramRepository`, etc.), allowing any implementation to be swapped without modifying callers. Controllers hold zero SQL and zero UI imports.
- **Testability:** Controllers are fully unit-testable with in-memory stub implementations. All domain objects are plain Java with no JavaFX dependency.
- **Extensibility:** New diagram types (Sequence, Activity) can be added by implementing `IDiagramRepository` with a new schema table and registering a new DiagramController method without touching existing code.
- **Portability:** The application is packaged as a fat JAR with `maven-shade-plugin` and runs on Windows, macOS, and Linux with JDK 17+.
- **Usability:** The diagram canvas replicates the draw.io interaction model (click-to-select, drag-to-move, right-click context menu) to minimise learning curve for existing diagramming tool users.
- **Reliability:** The ConnectionPool prevents concurrency deadlocks between the JavaFX Application Thread and background database threads.

3.5 Business Rules

- A project name must be unique within the system. Attempting to create or rename to a duplicate name throws a ValidationException and is rejected.
- A diagram must have at least 2 classes before the Design Quality Evaluation can be triggered (enforces DiagramTooSimpleException).
- An AI text description must be at least 10 characters long before generation is initiated (prevents trivial/noise requests).
- Orphan relationships (those referencing a deleted class) are automatically pruned by `pruneInvalidRelationships()` after every class removal operation.
- Only `.java` files may be uploaded for source-code-based diagram generation; other file types throw UnsupportedFileException.

3.6 Operating Environment

Component	Specification
Runtime	Java Development Kit (JDK) 17 or later
UI Framework	JavaFX 21 (OpenJFX)
Build Tool	Apache Maven 3.9+
Database	SQLite 3.45 via `org.xerial:sqlite-jdbc`
HTTP Client	OkHttp 4.12
JSON	Google Gson 2.10.1
Code Parsing	JavaParser 3.25.8
Export	Apache PDFBox, Apache Batik (SVG)
LLM API	Anthropic Claude / OpenAI GPT-4o (configurable via config.properties)
OS	Windows 10/11, macOS 12+, Ubuntu 22+
Min RAM	4 GB (8 GB recommended for large diagrams)

3.7 User Interfaces

UMLytics features five primary UI panels assembled by `MainWindow.java` as a JavaFX BorderPane:

- **Project Dashboard Panel:** Grid of project tiles showing name, last-modified date, and diagram count. Clicking a tile opens the project.
- **Diagram Editor Panel:** The central draw.io-replica canvas. Supports drag-to-place class nodes, click-to-select with bounding handles, right-click context menus for rename/delete, and a top toolbar for selecting the active relationship type (Association, Inheritance, Dependency).
- **AI Chat Panel (AIChatPanel):** A sidebar chat thread. User messages are right-aligned; AI responses are left-aligned with a bot icon. The full diagram state is serialised into the system prompt context on every AI message.
- **Evaluation Panel:** Displays three radial gauges (Coupling, Cohesion, SOLID) with colour-coded scores (green ≥ 0.7, yellow ≥ 0.5, red ¡ 0.5) and a scrollable text area for the AI feedback summary.
- **Project Explorer Panel:** A TreeView sidebar listing all diagrams under the open project. Double-click loads a diagram; right-click offers delete and rename options.

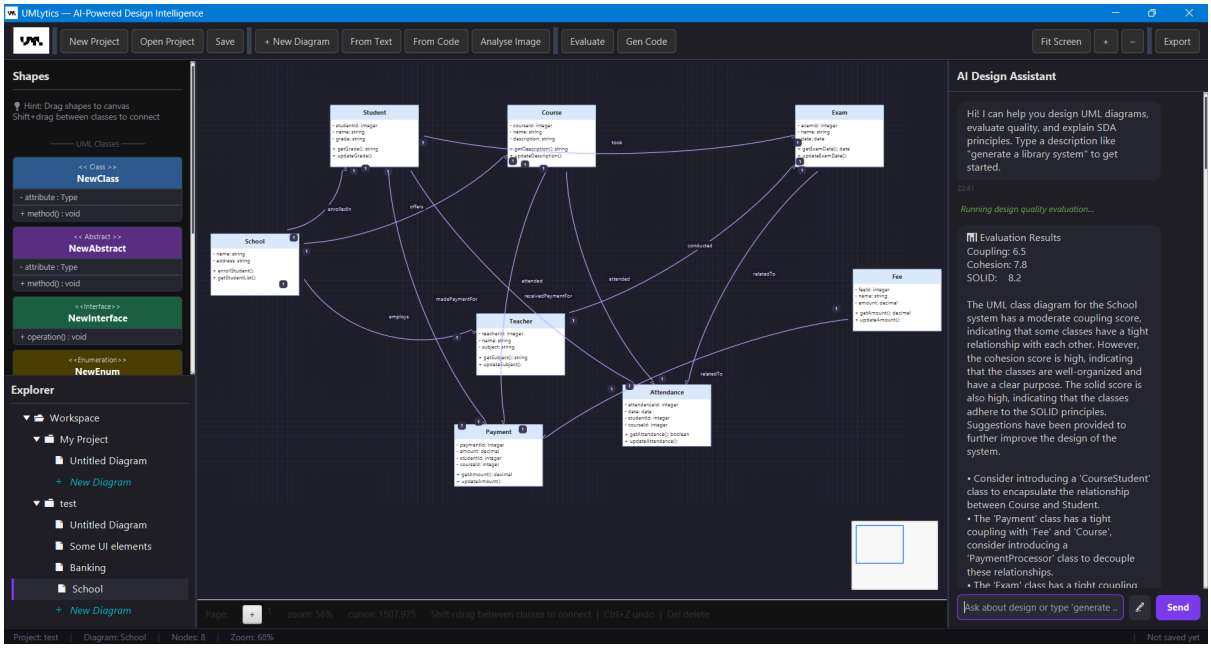

Figure 2: Main Application Window

4 Domain Model

The UMLytics domain model captures the key entities in the problem space and their conceptual relationships. It reflects the business vocabulary used across all twelve use cases.

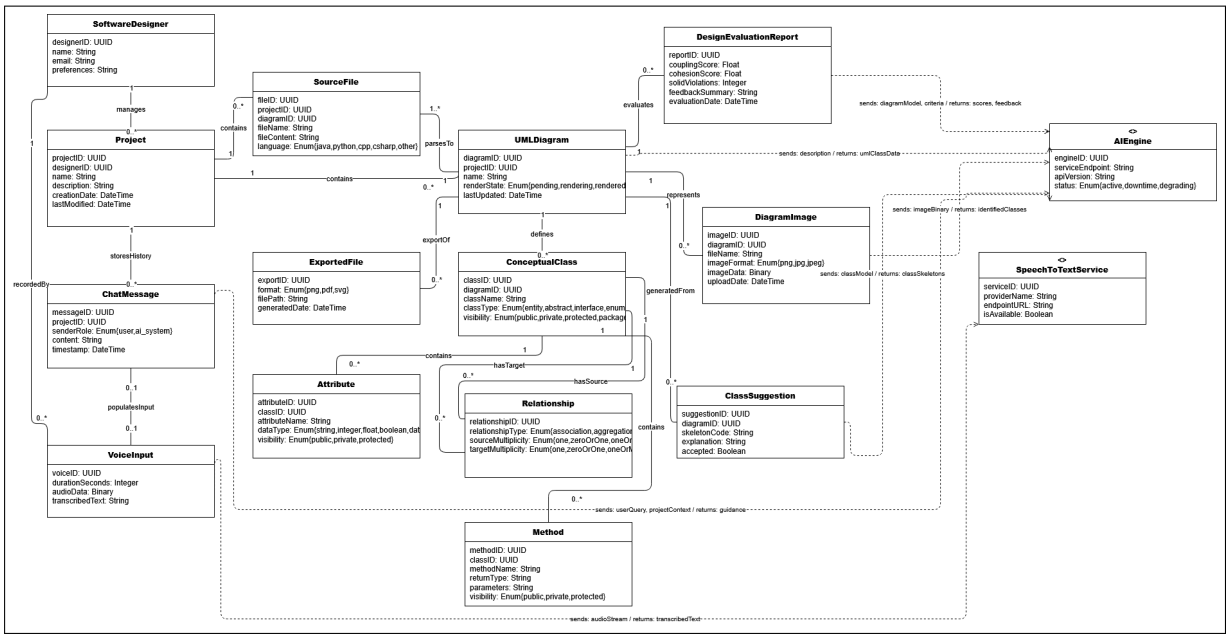

Figure 3: Domain Model

4.1 Key Domain Entities

Entity	Description
SoftwareDesigner	The primary user of the system. Creates projects, generates diagrams, and requests evaluations.
Project	A named workspace that groups one or more UML diagrams, chat messages, and evaluation reports.
UMLDiagram	A single class diagram artefact. Tracks source type (NL/Code/Manual/Image), render state, and last-updated timestamp.
ConceptualClass	A UML class node. Holds a list of Attributes and Methods, plus visual metadata (position, color, size).
Attribute	A typed field belonging to a ConceptualClass. Has visibility, name, data type, and optional default value.
Method	A typed operation belonging to a ConceptualClass. Has visibility, return type, name, and parameter list.
Relationship	An abstract edge between two ConceptualClasses. Subtyped by Association, Inheritance, and Dependency.
DesignEvaluationReport	The AI-generated quality assessment: coupling score, cohesion score, SOLID score, and a text feedback summary.
ChatMessage	A single turn in the AI co-pilot conversation, tagged with SenderType (USER / AI / SYSTEM).
ClassSuggestion	AI-generated Java skeleton code for one or more classes in the diagram.
DiagramImage	Metadata for an uploaded PNG/JPG image associated with an image-sourced diagram.
SourceFile	A Java source file uploaded for reverse-engineering. Stores content and detected language.

4.2 Key Domain Relationships

- A SoftwareDesigner creates one or more Projects.
- A Project contains zero or more UMLDiagrams, ChatMessages, and DesignEvaluationReports.
- A UMLDiagram aggregates one or more ConceptualClasses and zero or more Relationships.
- A ConceptualClass composes one or more Attributes and one or more Methods.
- A Relationship connects exactly two ConceptualClasses (source and target) and is contained by a UMLDiagram.
- A DesignEvaluationReport is generated from exactly one UMLDiagram and belongs to one Project.
- A ClassSuggestion may be scoped to a single ConceptualClass or to an entire UMLDiagram.

5 System Sequence Diagrams

System Sequence Diagrams (SSDs) model the messages exchanged between the primary actor and the system boundary for each use case. They treat the entire UMLytics application as a black box and show inputs and outputs at the system level.

5.1 SSD1 — Create / Initialize Project

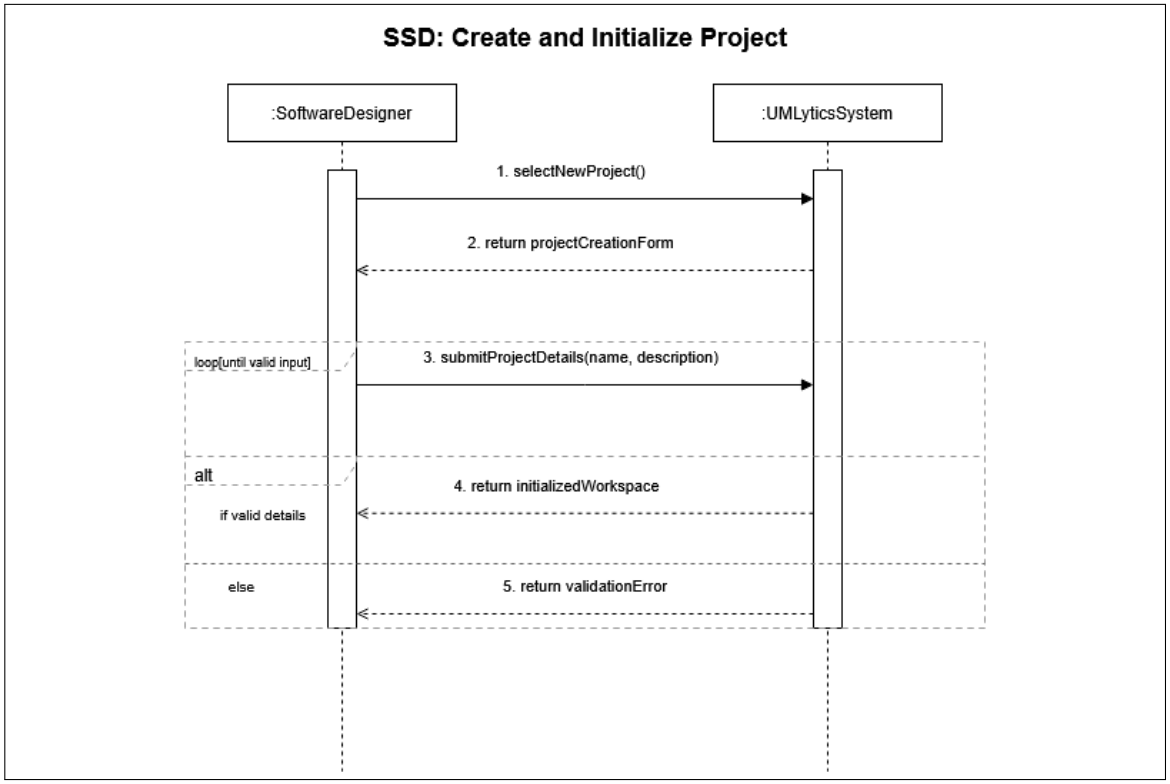

Figure 4: System Sequence Diagram: Create / Initialize Project

5.2 SSD2 — Generate Diagram from Natural Language

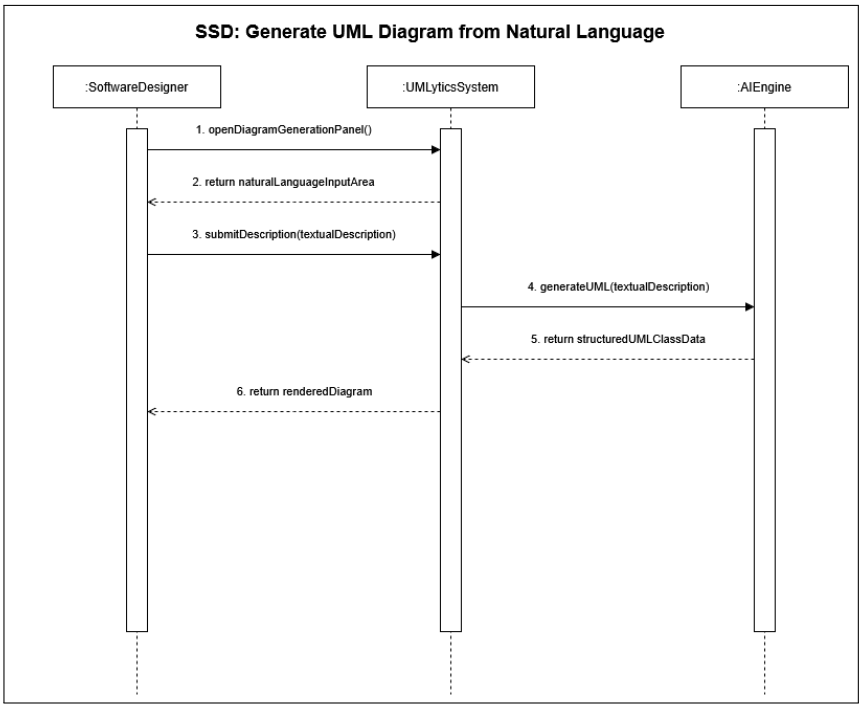

Figure 5: System Sequence Diagram: Generate Diagram from Natural Language

5.3 SSD3 — Generate Diagram from Source Code

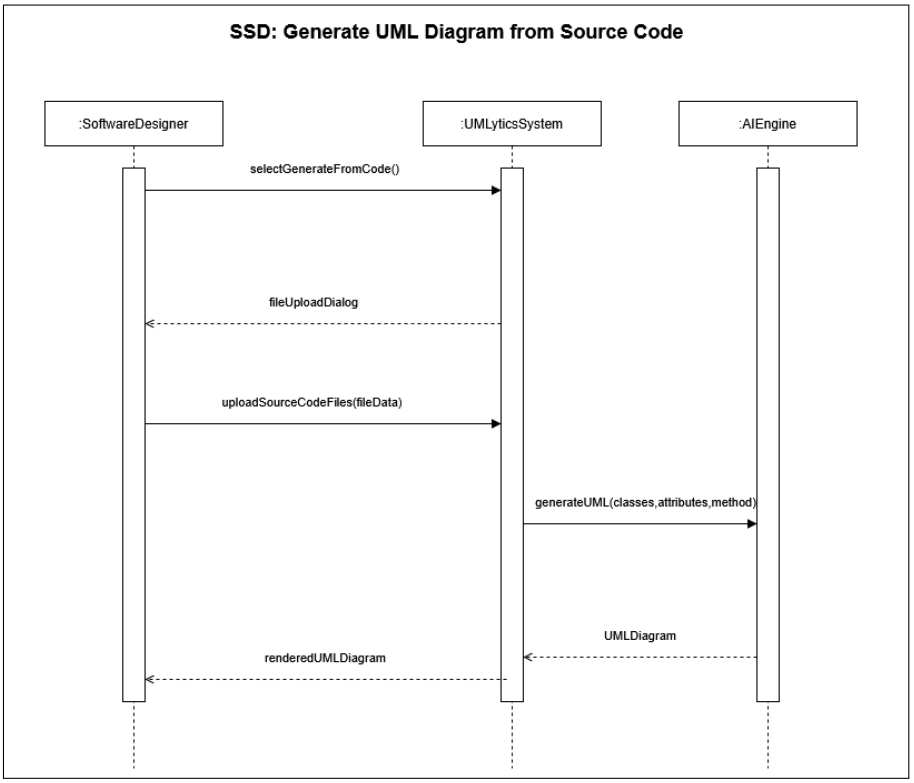

Figure 6: System Sequence Diagram: Generate Diagram from Source Code

5.4 SSD4 — Manually Edit Diagram

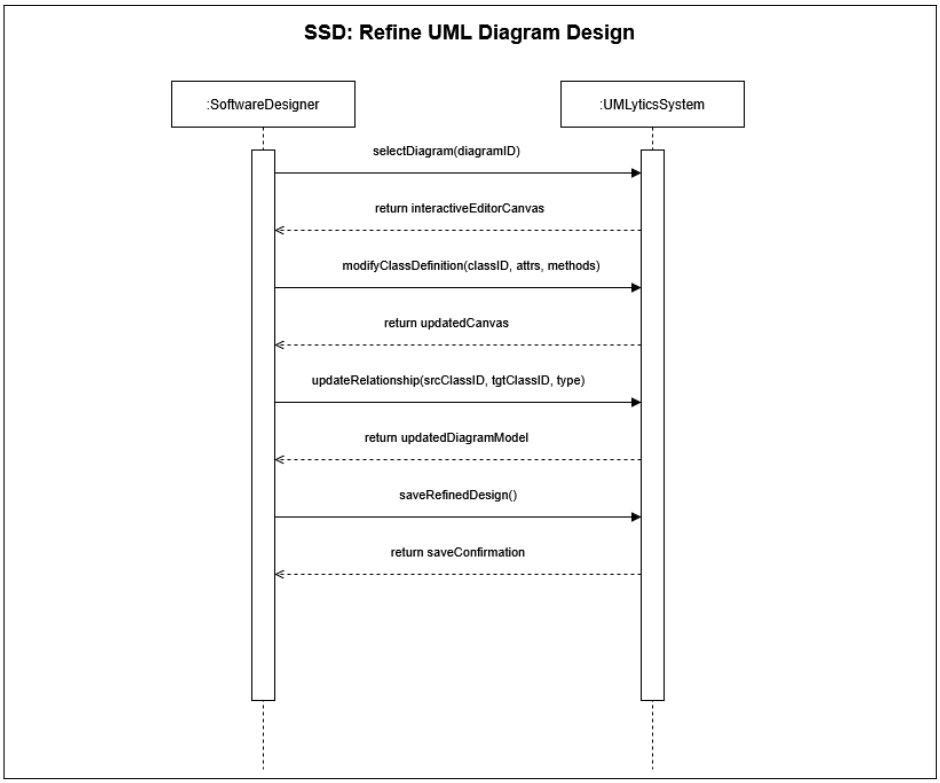

Figure 7: System Sequence Diagram: Manually Edit Diagram

5.5 SSD5 — Evaluate Design Quality

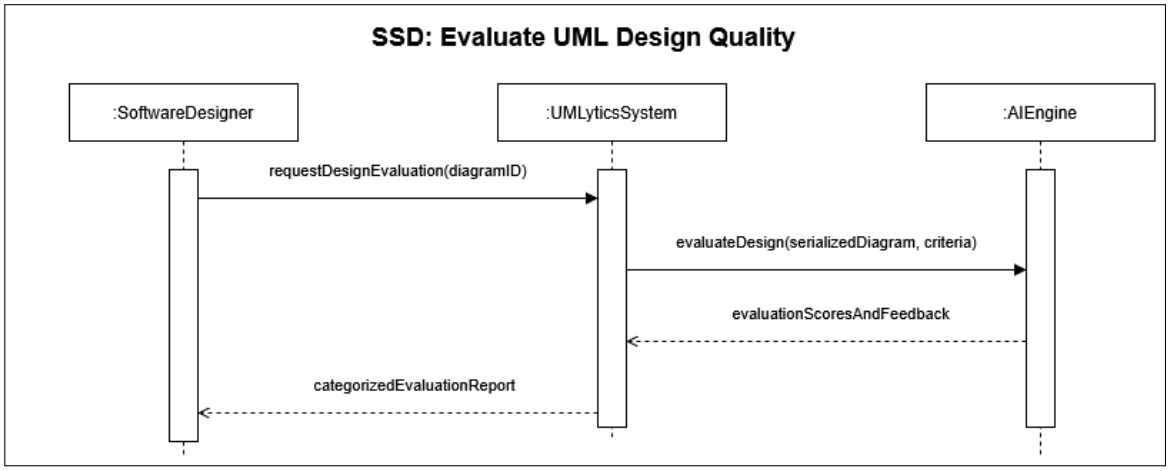

Figure 8: System Sequence Diagram: Evaluate Design Quality

5.6 SSD6 — Consult AI Design Guidance

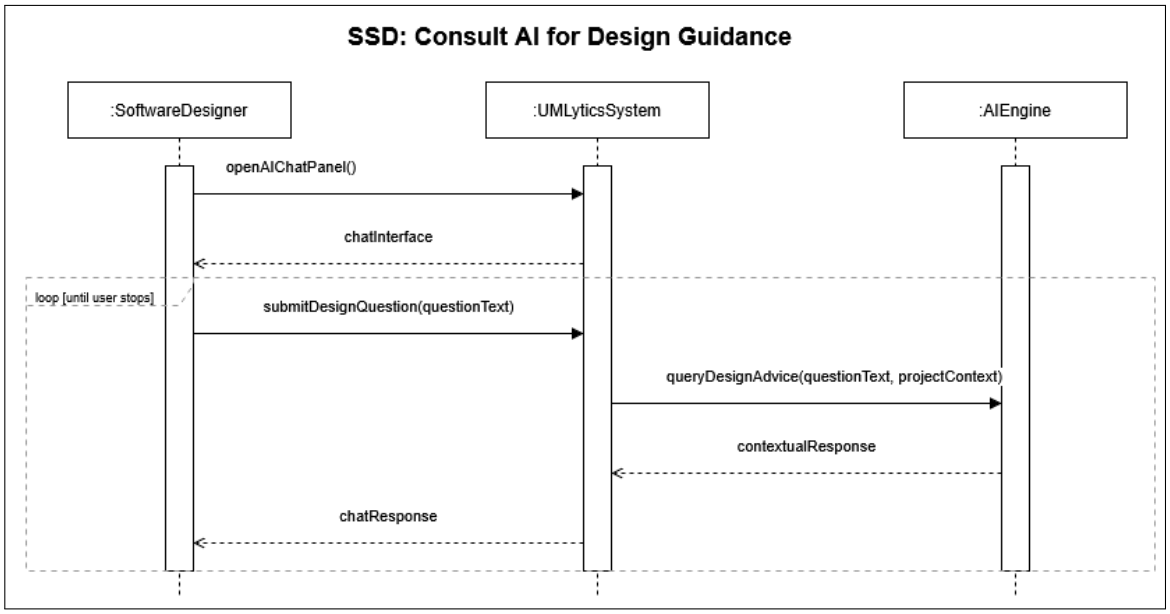

Figure 9: System Sequence Diagram: Consult AI Design Guidance

5.7 SSD7 — Generate Class Suggestions

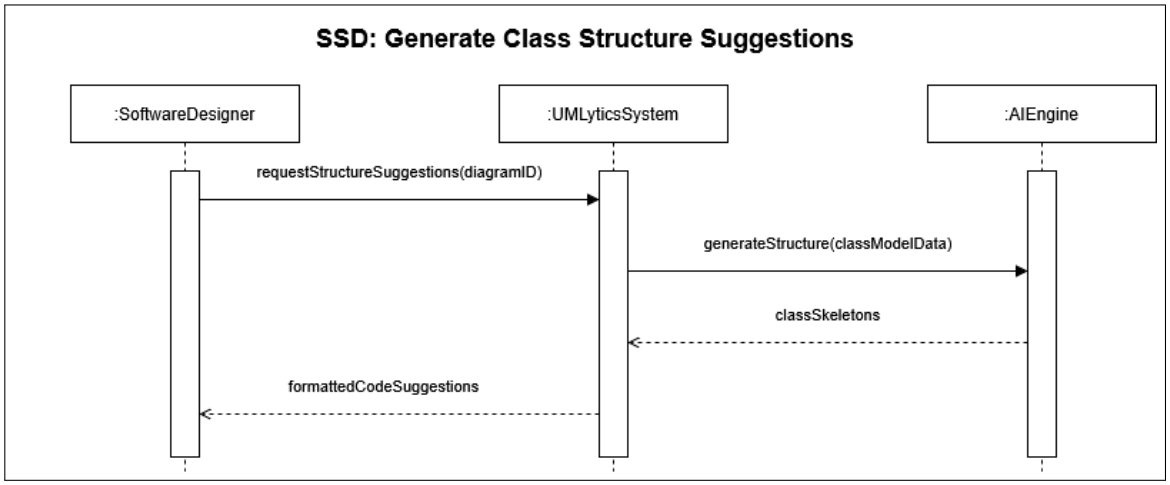

Figure 10: System Sequence Diagram: Generate Class Suggestions

5.8 SSD8 — Analyze Uploaded Diagram Image

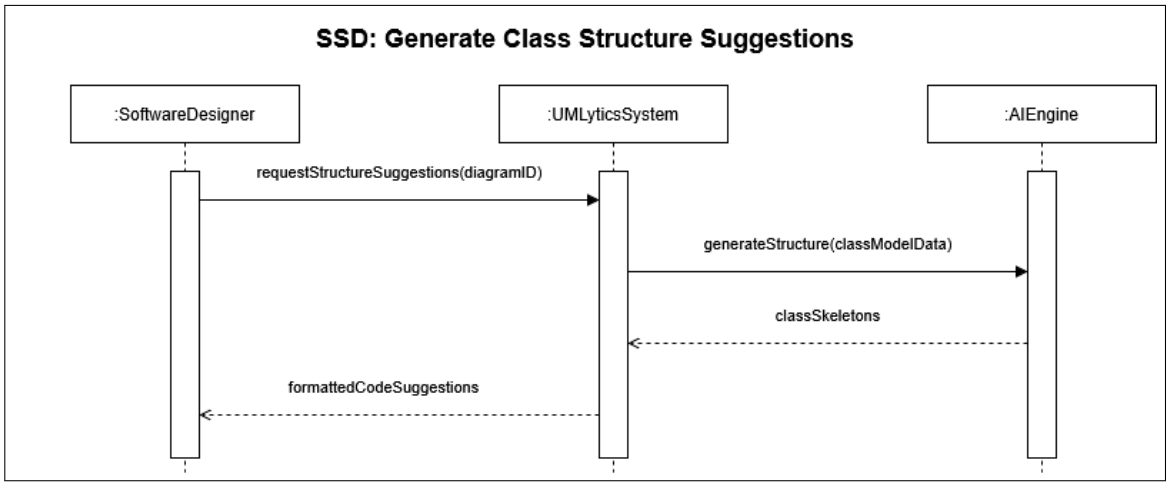

Figure 11: System Sequence Diagram: Analyze Uploaded Diagram Image

5.9 SSD9 — Maintain Project Data

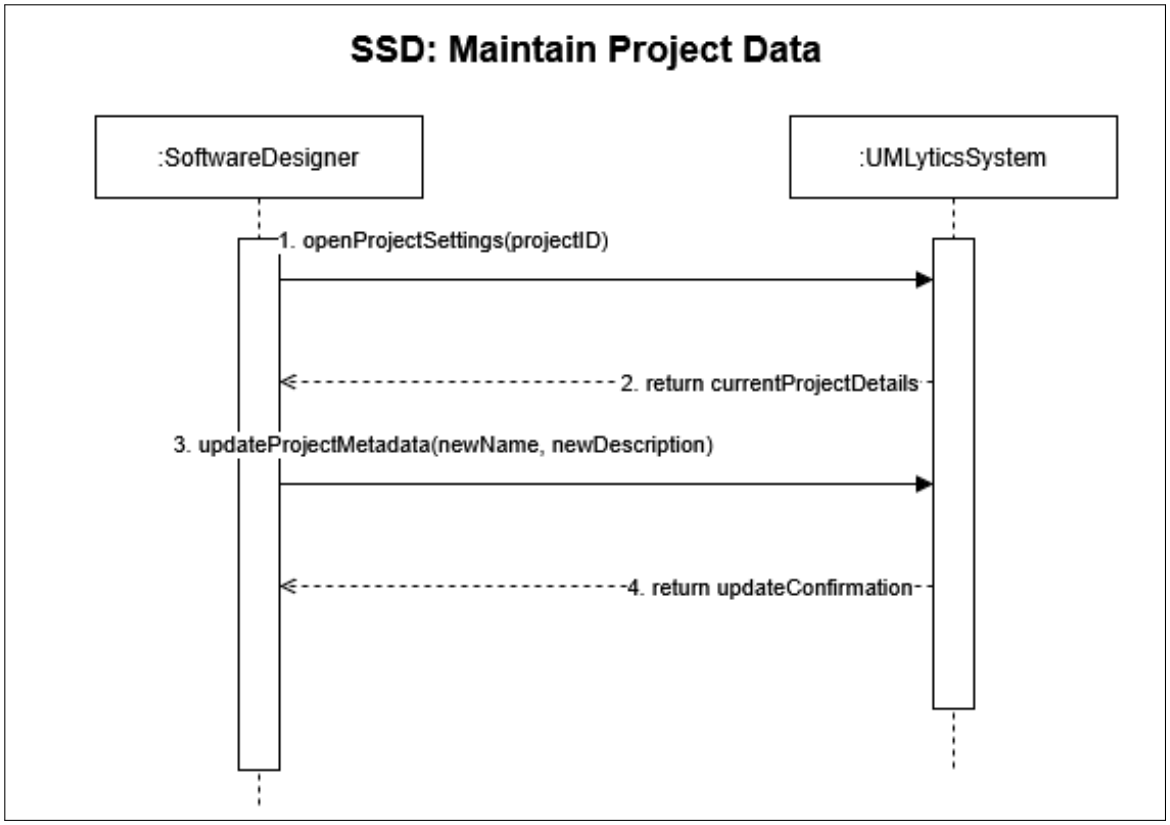

Figure 12: System Sequence Diagram: Maintain Project Data

5.10 SSD10 — Retrieve Existing Project

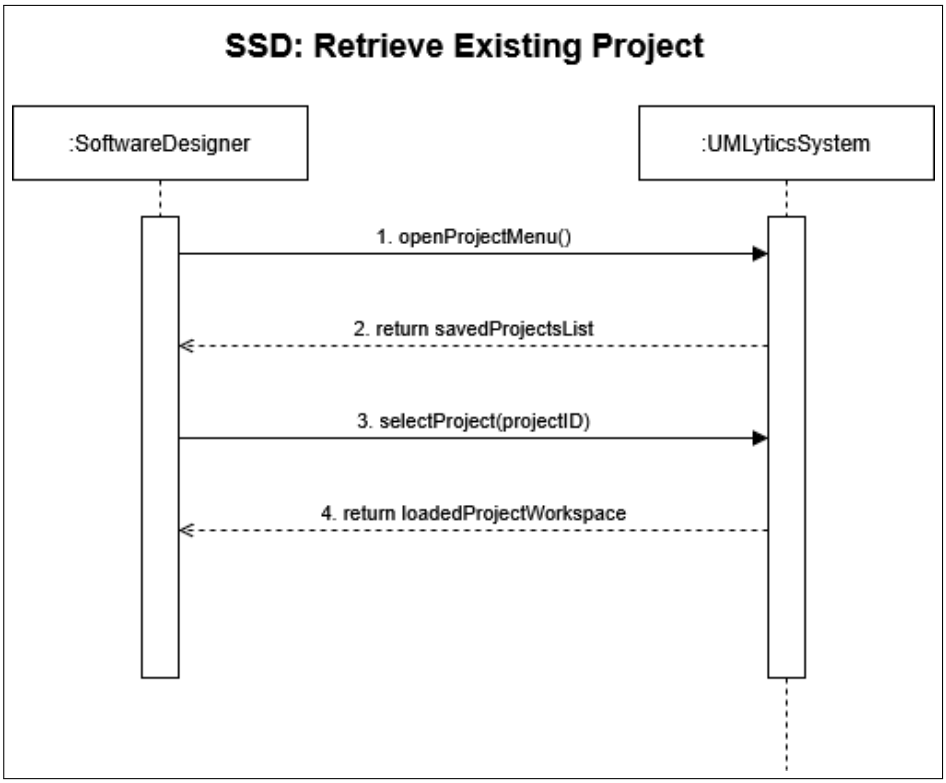

Figure 13: System Sequence Diagram: Retrieve Existing Project

5.11 SSD11 — Voice-Based Input

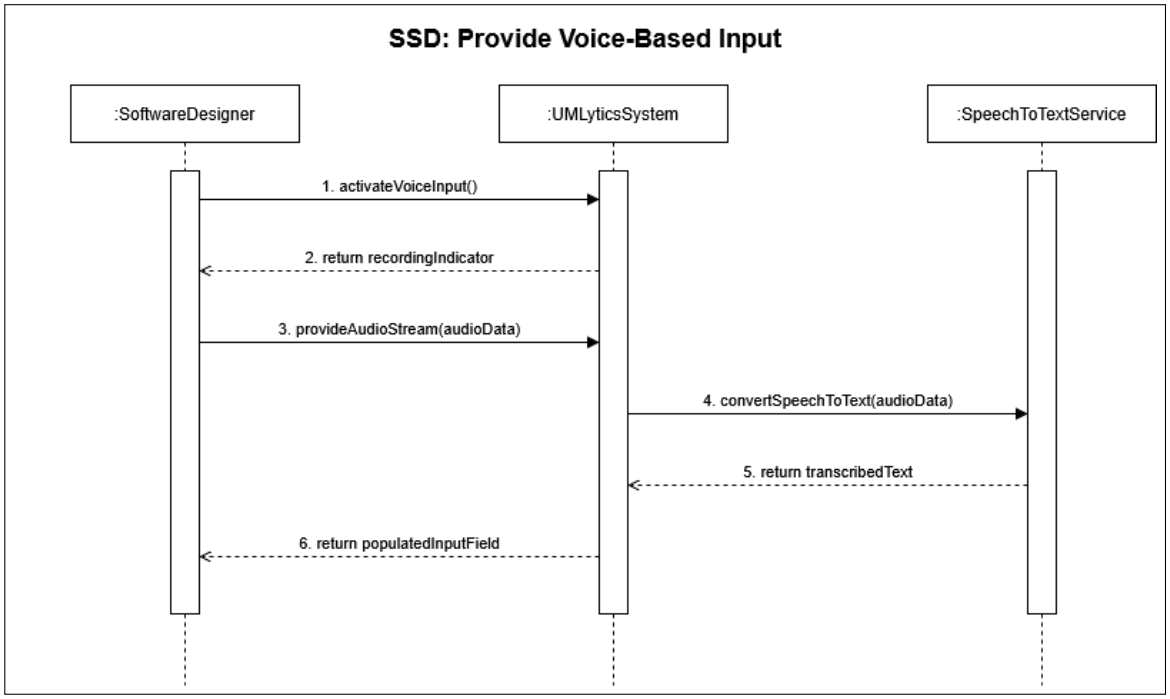

Figure 14: System Sequence Diagram: Voice-Based Input

5.12 SSD12 — Export UML Diagram

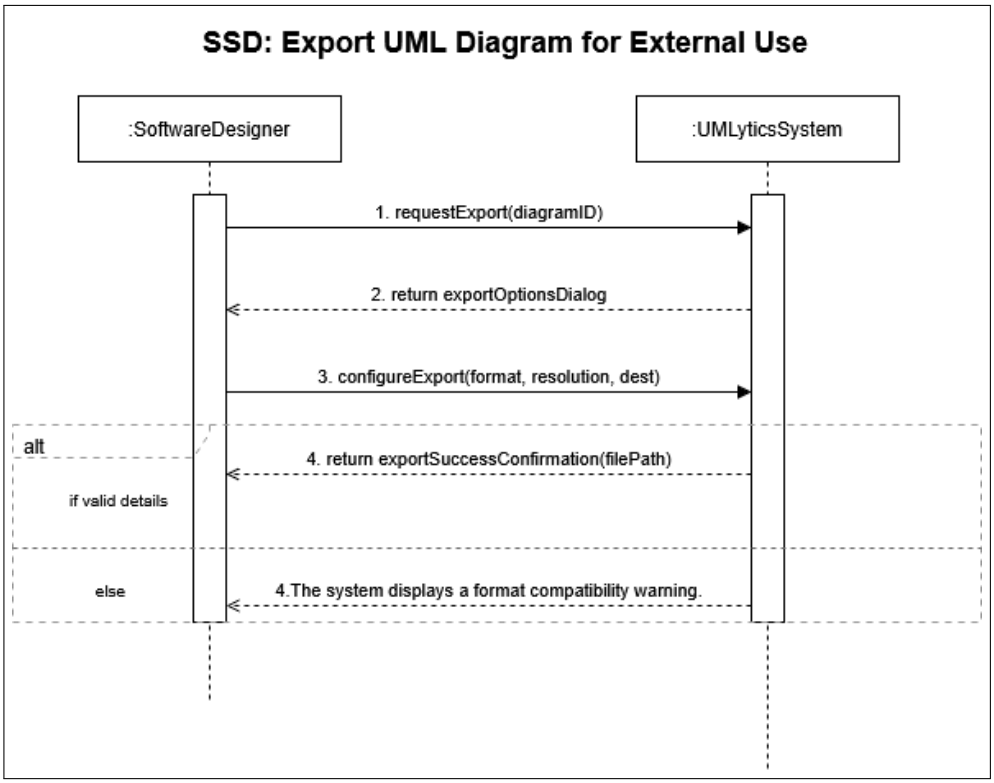

Figure 15: System Sequence Diagram: Export UML Diagram

6 Sequence Diagrams (Design Level)

Design-level Sequence Diagrams describe the internal object interactions for each use case, including the specific method calls between the Controller, Service, Repository, and Domain layers.

6.1 SD1 — Create / Initialize Project

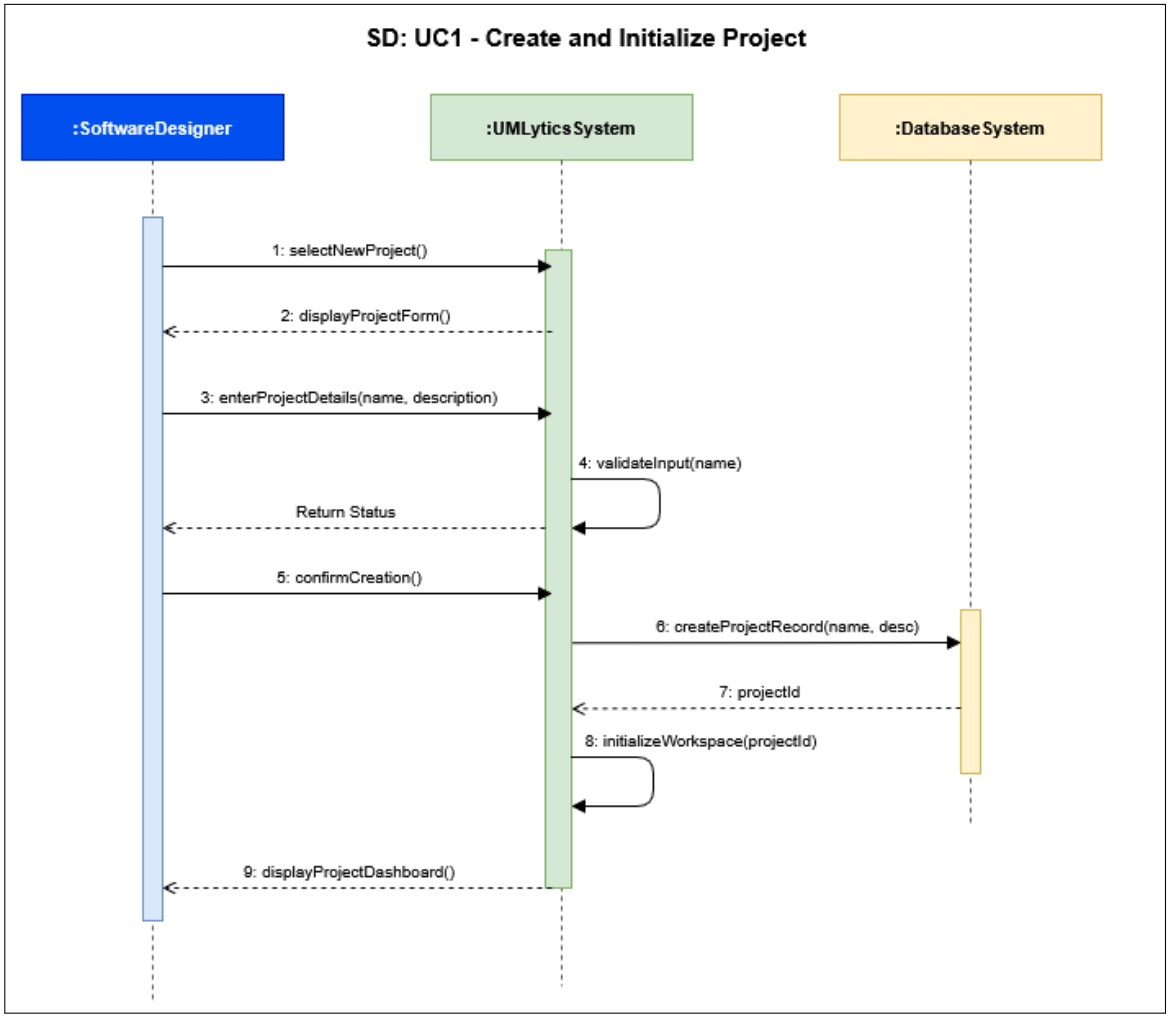

Figure 16: Sequence Diagram: Create / Initialize Project

6.2 SD2 — Generate Diagram from Natural Language

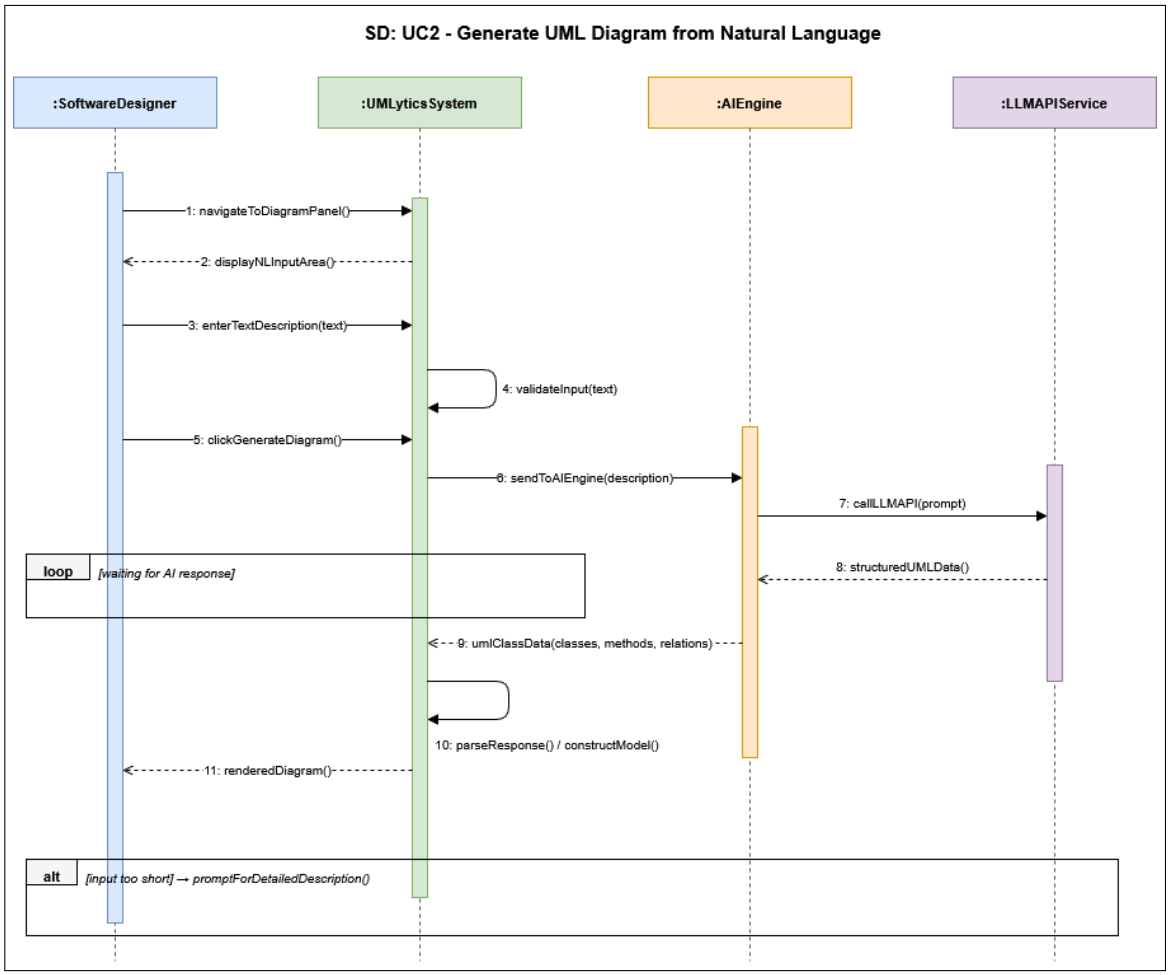

Figure 17: Sequence Diagram: Generate Diagram from Natural Language

6.3 SD3 — Generate Diagram from Source Code

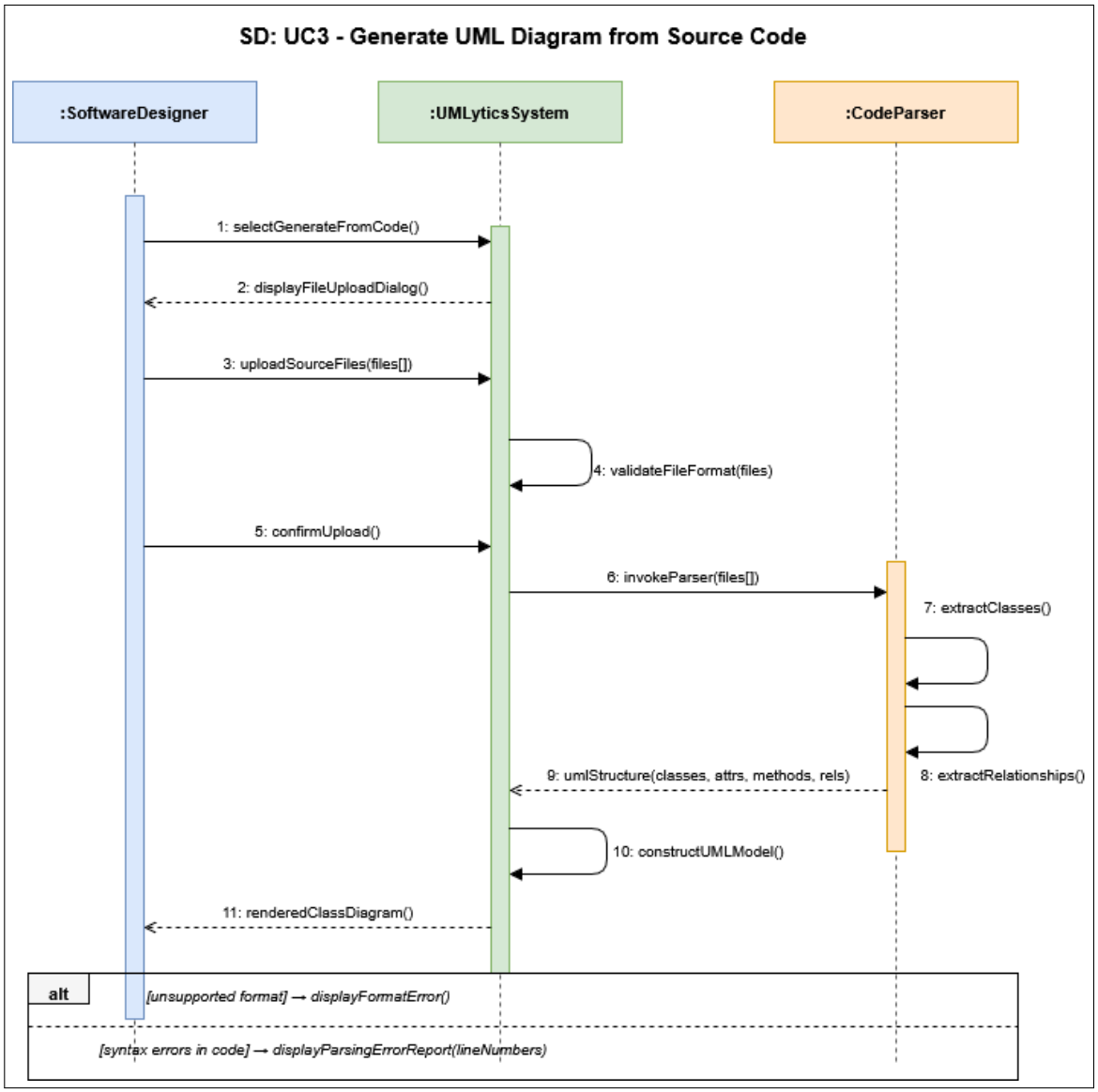

Figure 18: Sequence Diagram: Generate Diagram from Source Code

6.4 SD4 — Manually Edit Diagram

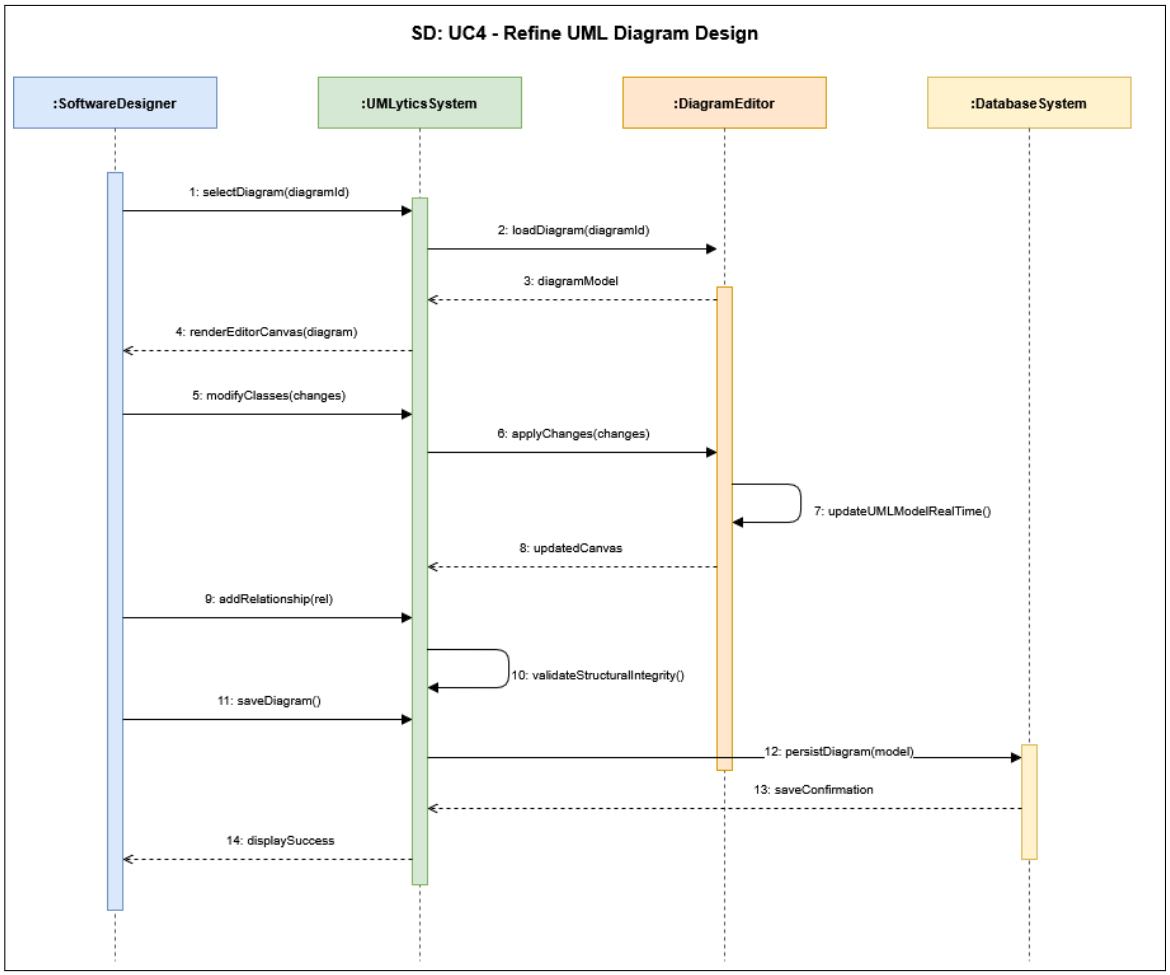

Figure 19: Sequence Diagram: Manually Edit Diagram

6.5 SD5 — Design Quality Evaluation

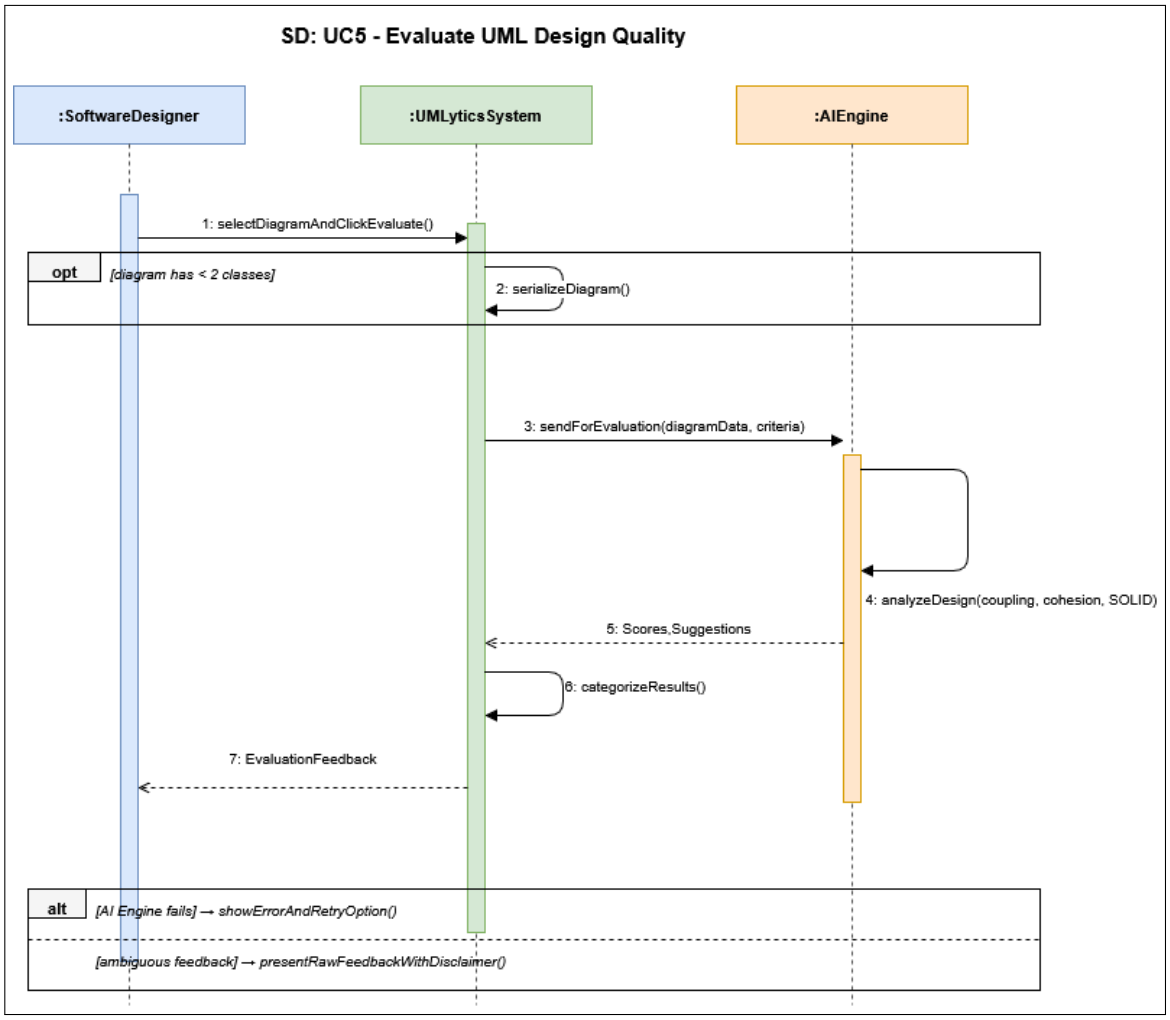

Figure 20: Sequence Diagram: Design Quality Evaluation

6.6 SD6 — Consult AI Design Guidance

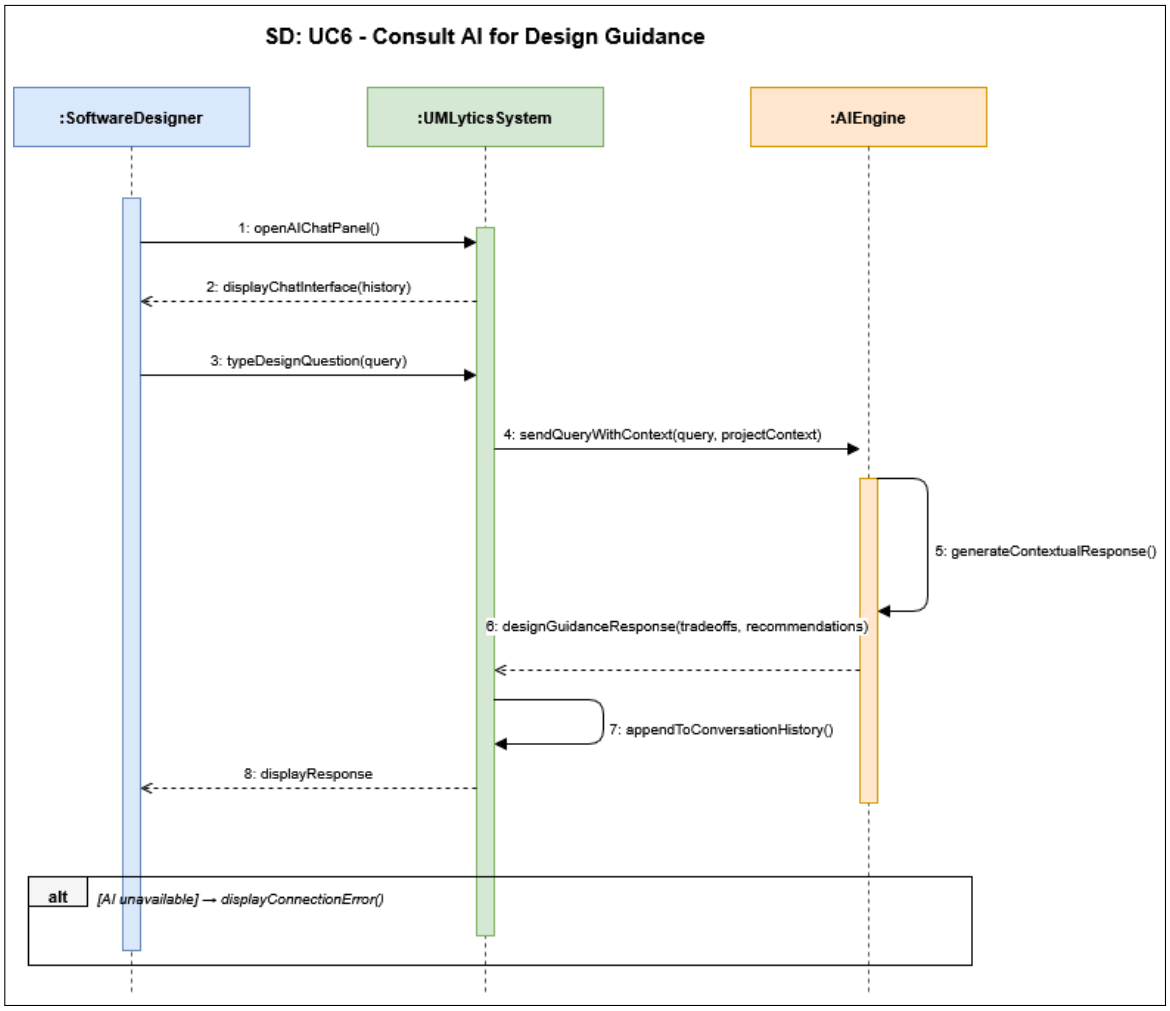

Figure 21: Sequence Diagram: Consult AI Design Guidance

6.7 SD7 — Generate Class Suggestions

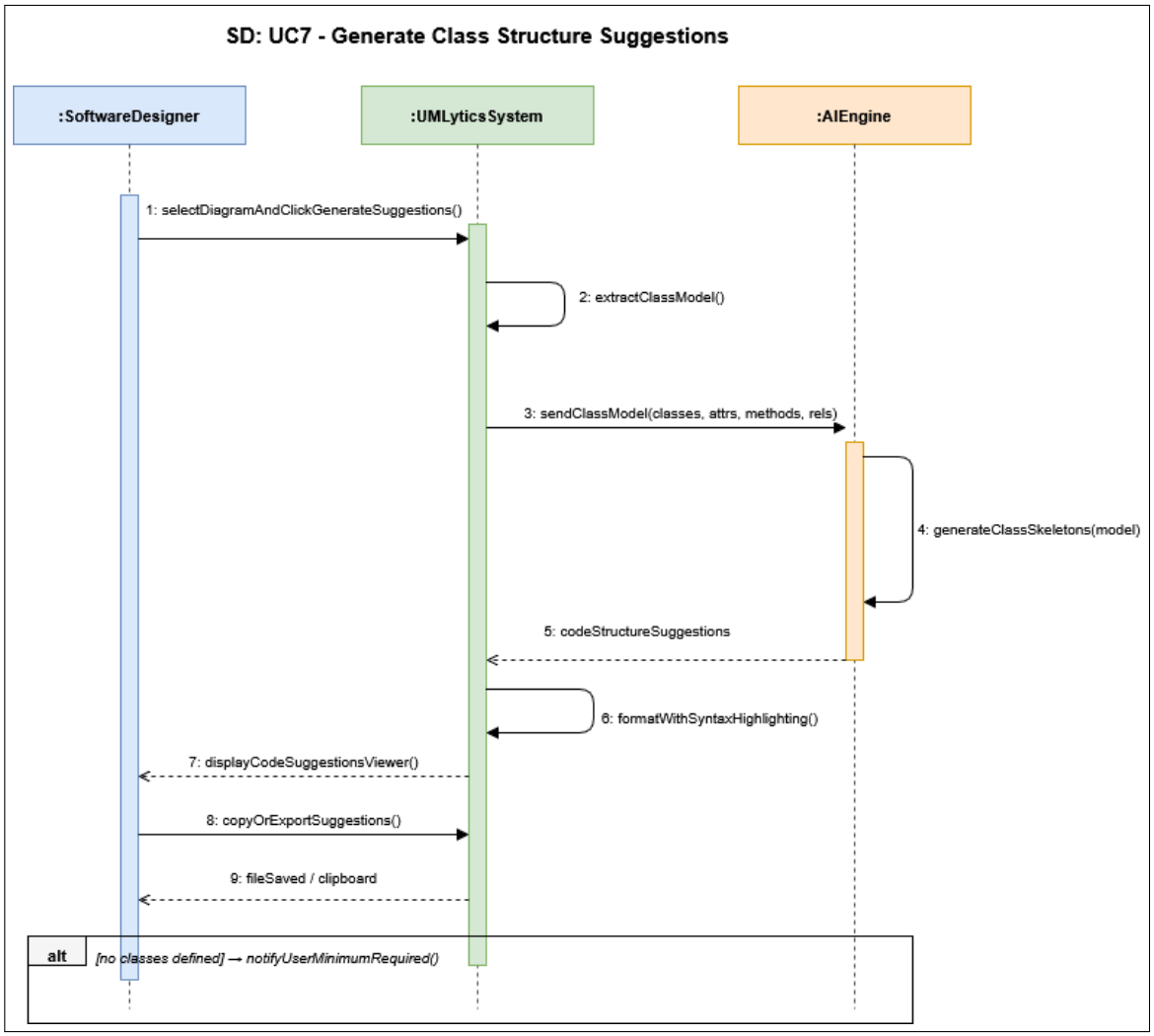

Figure 22: Sequence Diagram: Generate Class Suggestions

6.8 SD8 — Analyze Uploaded Diagram Image

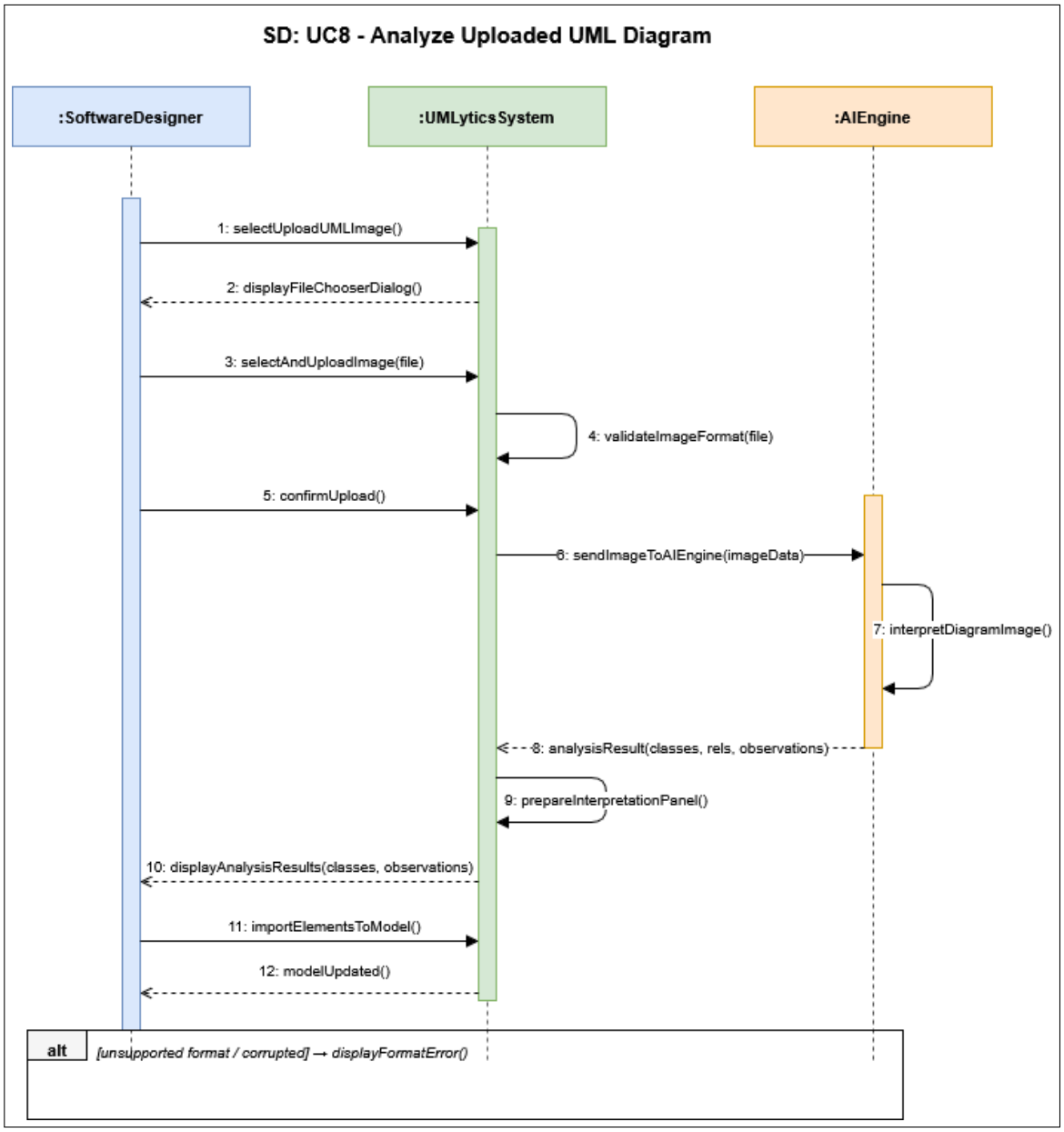

Figure 23: Sequence Diagram: Analyze Uploaded Diagram Image

6.9 SD9 — Maintain Project Data

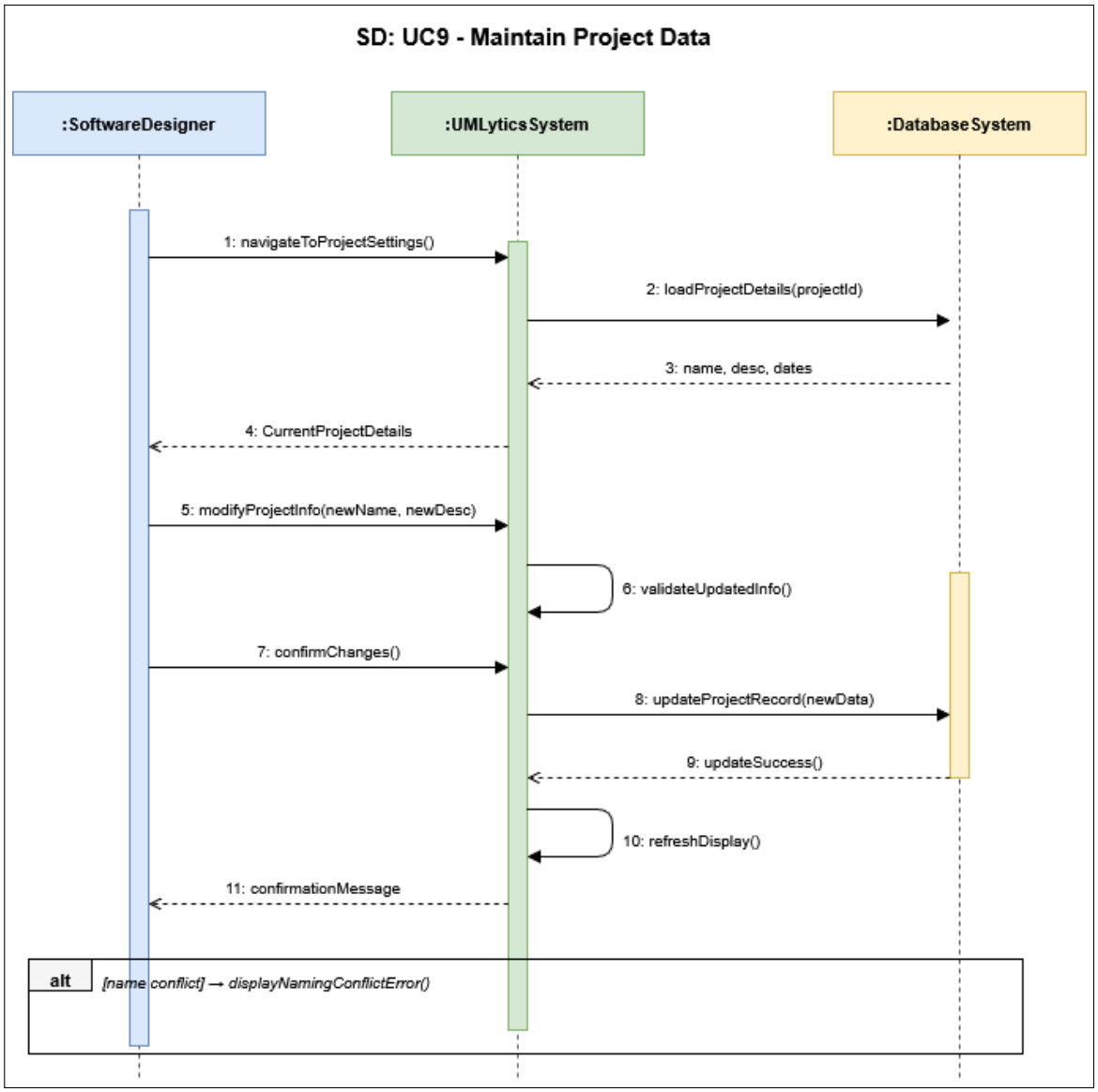

Figure 24: Sequence Diagram: Maintain Project Data

6.10 SD10 — Retrieve Existing Project

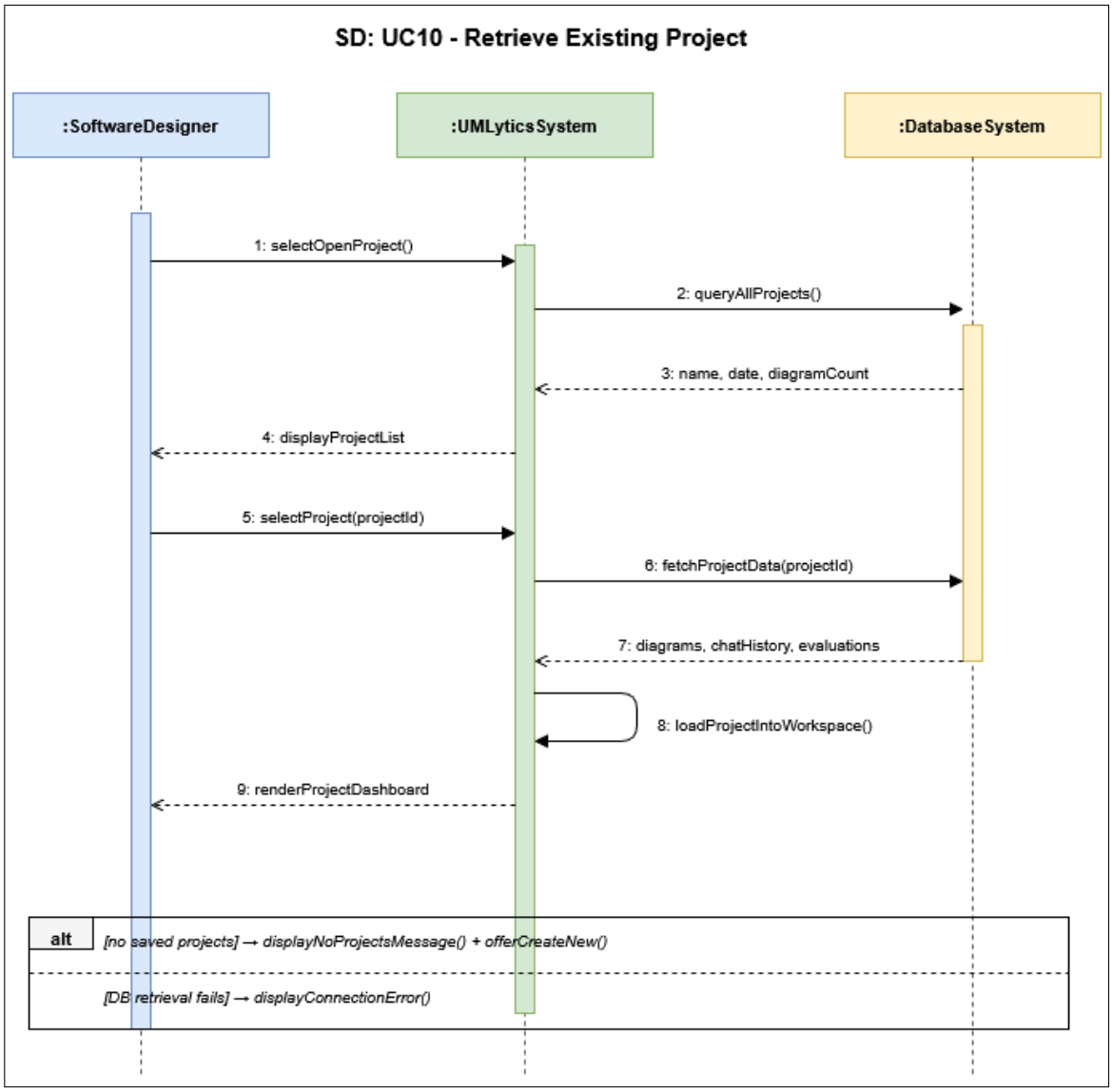

Figure 25: Sequence Diagram: Retrieve Existing Project

6.11 SD11 — Voice-Based Input

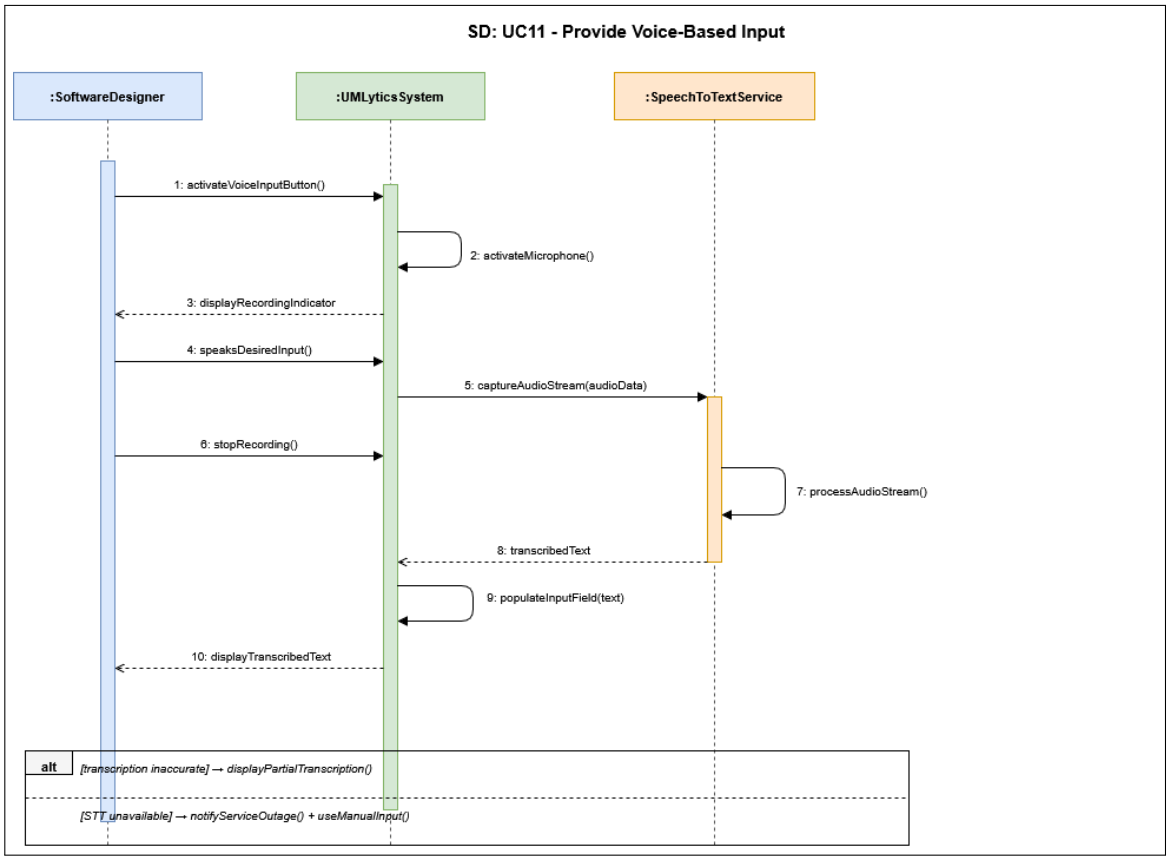

Figure 26: Sequence Diagram: Voice-Based Input

6.12 SD12 — Export UML Diagram

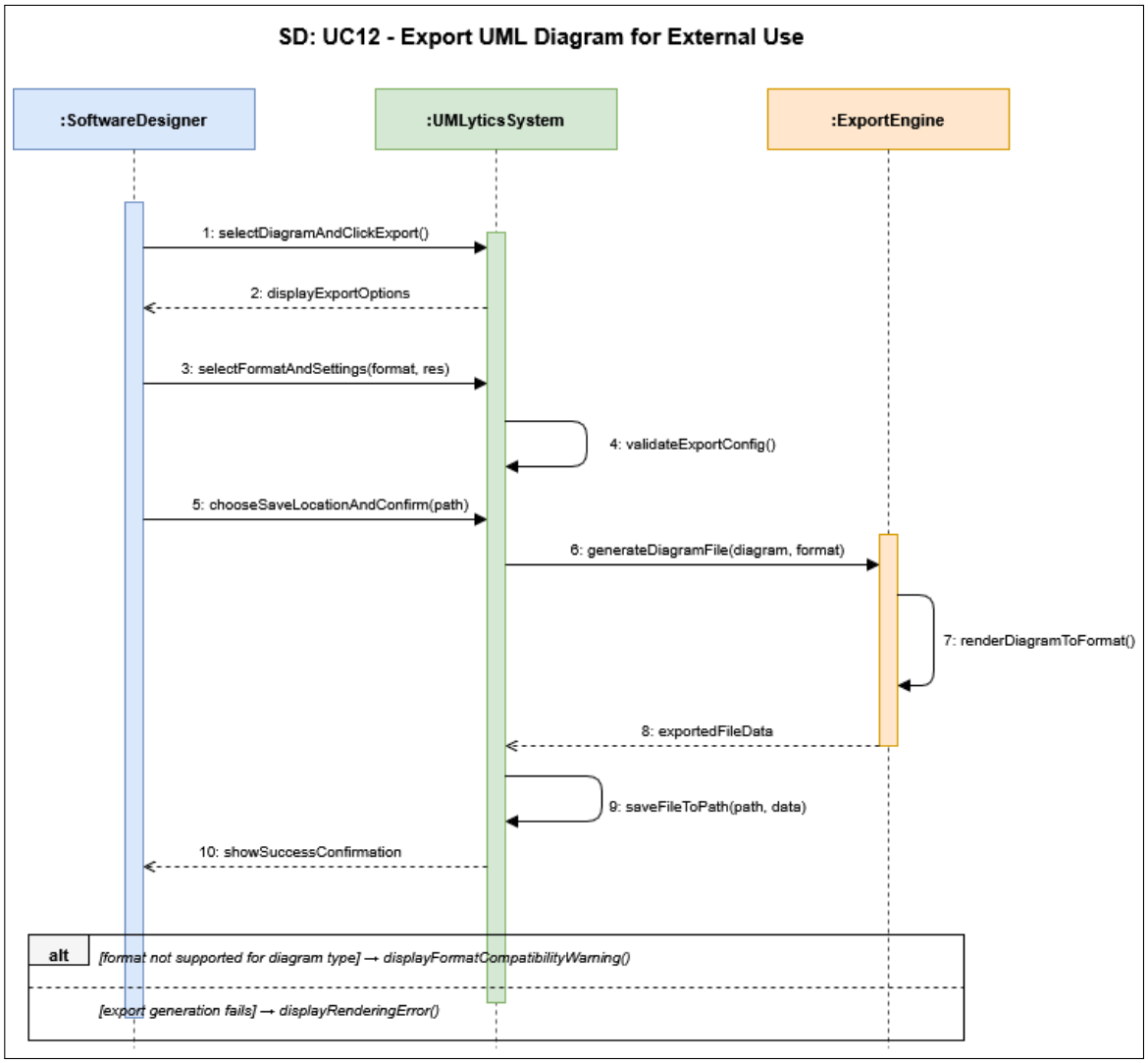

Figure 27: Sequence Diagram: Export UML Diagram

7 Class Diagram

The UMLytics class diagram reflects the complete production codebase under `src/main/java/com/umlytics`. It enforces GRASP principles (Controller, Creator, Pure Fabrication, Polymorphism) and the GoF Singleton and Facade patterns.

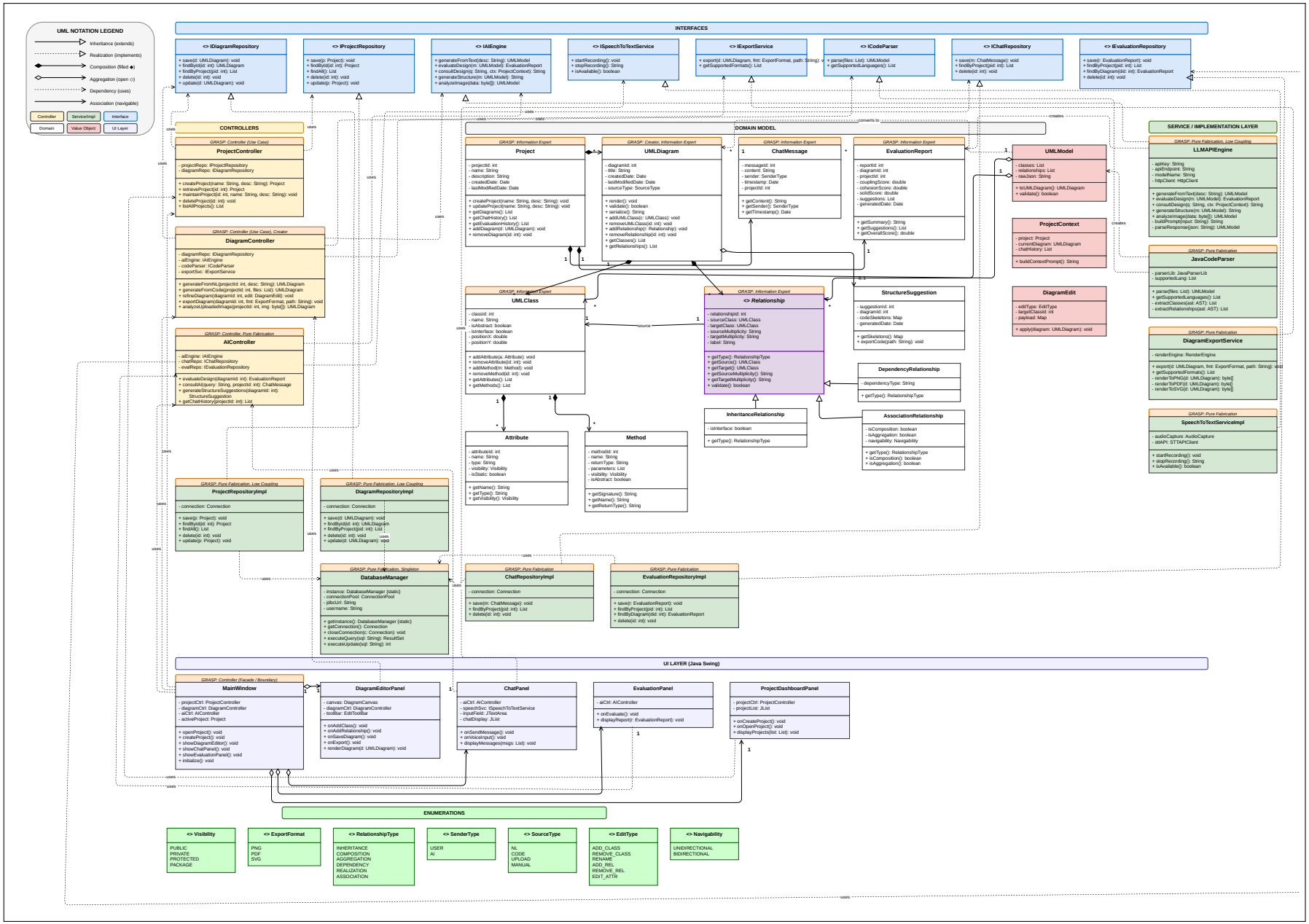

Figure 28: Class Diagram

7.1 Layer Overview

Layer	Package	Key Classes
Presentation	com.umlytics.ui	MainWindow, DiagramEditorPanel, AIChatPanel, EvaluationPanel, ProjectDashboardPanel
Canvas	com.umlytics.ui.canvas	ClassNode, RelationshipEdge, SelectionHandle, EditToolBar, DiagramCanvas
Controller	com.umlytics.controllers	DiagramController, AIController, ProjectController
Domain	com.umlytics.domain	UMLDiagram, ConceptualClass, Attribute, Method, Relationship, DesignEvaluationReport, ChatMessage, DiagramEdit, UMLModel, ProjectContext
Service	com.umlytics.services	LLMAPIEngine, JavaCodeParser, DiagramExportService, SpeechToTextServiceImpl, AudioCapture, RenderEngine
Repository	com.umlytics.repository	DiagramRepositoryImpl, ProjectRepositoryImpl, ChatRepositoryImpl, DesignEvaluationRepositoryImpl
Database	com.umlytics.db	DatabaseManager (Singleton), ConnectionPool
Interfaces	com.umlytics.interfaces	IAIEngine, IDiagramRepository, IProjectRepository, ICodeParser, IExportService, ISpeechToTextService, IChatRepository, IEvaluationRepository
Enums	com.umlytics.enums	Visibility, RelationshipType, EditType, ExportFormat, SourceType, ClassType, SenderType, RenderState, Navigability, ImageFormat
Exceptions	com.umlytics.exceptions	ValidationException, AIEngineException, DatabaseException, DiagramTooSimpleException, EmptyDiagramException, UnsupportedFileException, ParseResponseException

7.2 Key Design Patterns Applied

- **Singleton — DatabaseManager:** DatabaseManager is implemented as a strict Singleton. Its static `getInstance()` method performs double-checked locking to ensure only one JDBC connection hub exists across all repository threads.
- **Facade — MainWindow:** MainWindow acts as the GoF Facade for the entire UI layer. It bootstraps the CSS theme, assembles the BorderPane with all sub-panels, maps keyboard accelerators (Ctrl+S for save, Ctrl+Z for undo), and provides the shared `showToast(message)` overlay mechanism. No sub-panel directly instantiates another sub-panel.
- **Polymorphism — Relationship Hierarchy:** The abstract base class `Relationship` is extended by `AssociationRelationship`, `InheritanceRelationship`, and

`DependencyRelationship`. Each subclass overrides line styling, multiplicity defaults, and arrow-head rendering without any if-statement type-checking in the canvas code.

- **GRASP Controller — DiagramController, AIController, ProjectController:** The three controller classes receive all UI events and system operations. They contain no JavaFX imports and no SQL strings—they delegate to interfaces and domain methods exclusively, making them independently unit-testable.
- **Pure Fabrication — Repository Implementations:** The repository classes (DiagramRepositoryImpl, etc.) do not correspond to any real-world domain concept. They exist purely to isolate JDBC code from the controllers, a textbook example of GRASP Pure Fabrication.

8 Package Diagram

The package diagram illustrates the dependency relationships between the eight major packages in the UMLytics codebase. Dependencies flow strictly downward: the UI layer depends on Controllers; Controllers depend on Interfaces; Interfaces are implemented by Services and Repositories; both depend on the Domain and DB packages.

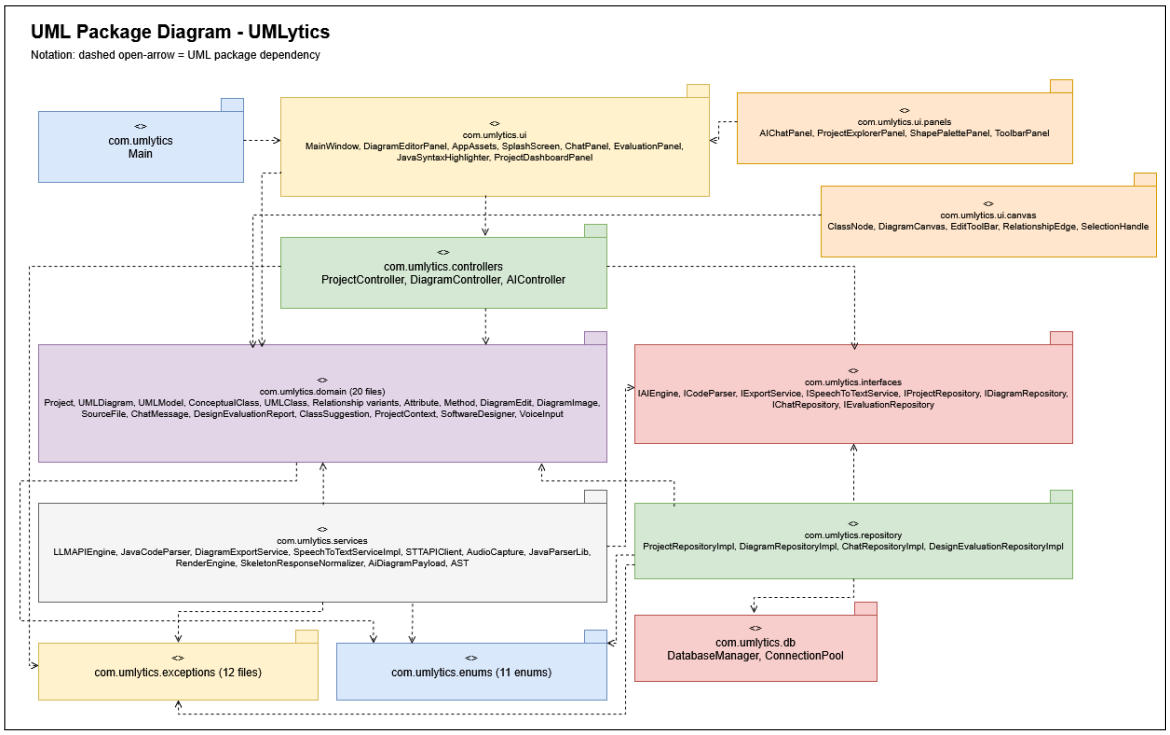

Figure 29: Package Diagram

8.1 Package Dependency Rules

- `com.umlytics.ui` depends on: controllers, domain, enums, exceptions
- `com.umlytics.controllers` depends on: interfaces, domain, enums, exceptions
- `com.umlytics.services` depends on: interfaces, domain, enums, exceptions
- `com.umlytics.repository` depends on: interfaces, domain, db, exceptions
- `com.umlytics.domain` depends on: enums only (no upward dependencies)

- `com.umlytics.db` depends on: exceptions only
- `com.umlytics.interfaces` depends on: domain, enums only
- `com.umlytics.exceptions` has no dependencies on other umlytics packages

9 Deployment Diagram

The deployment diagram describes the physical distribution of UMLytics software artefacts across execution nodes.

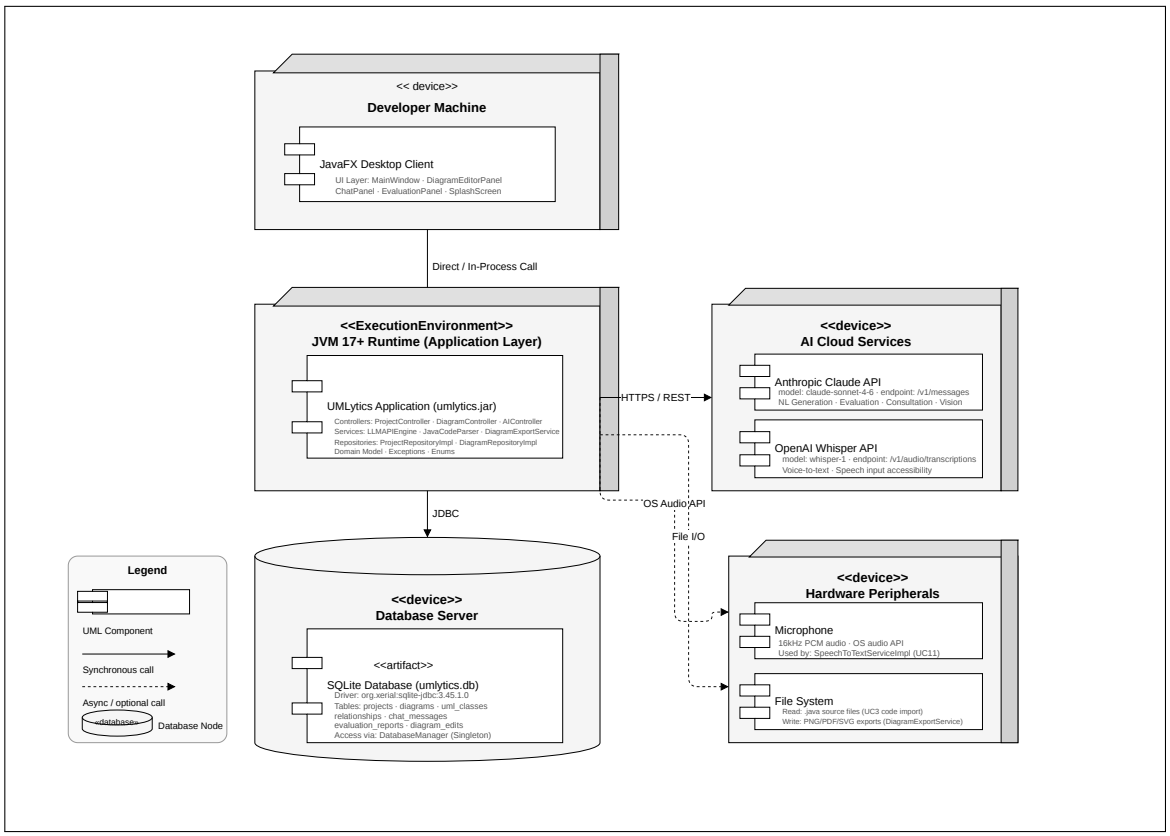

Figure 30: Deployment Diagram

9.1 Deployment Nodes

Node	Description
Developer Workstation	Windows/macOS/Linux machine running JDK 17+. Hosts the UMLytics JAR, the SQLite .db file, and config.properties. All local I/O (file parsing, export) occurs here.
JVM Process	The Java Virtual Machine executing `Main.java` → `Application.launch(MainWindow)`. Manages all JavaFX rendering, background executor threads, and ConnectionPool.
SQLite File	A single `umlytics.db` file co-located with the JAR. Accessed via `org.xerial:sqlite-jdbc` through DatabaseManager. No network port required.
LLM API Server	Anthropic Claude or OpenAI GPT-4o cloud endpoint. Accessed over HTTPS by LLMAPIEngine via OkHttp. Not hosted by the team; requires a valid API key in config.properties.
STT API Server	Speech-to-text cloud endpoint (configurable). Accessed by STTAPIClient. Only active when the microphone input feature is used (UC11).

9.2 Communication Protocols

- JVM Process → LLM API Server: HTTPS (TLS 1.3), JSON payloads, OkHttp 4.12
- JVM Process → SQLite File: Local JDBC file I/O (no network)
- JVM Process → STT API Server: HTTPS, PCM audio stream
- JVM Process → Local Filesystem: Direct file I/O for source file parsing and diagram export

A Key Code Snippets

A.1 DiagramController — generateFromText

```
// GRASP: Controller - mediates between UI and AI/Repository layers
public UMLDiagram generateFromText(UUID projectId, String desc) {
    if (desc == null || desc.trim().length() <= 10) {
        throw new ValidationException("Description too short.");
    }
    UMLModel model = aiEngine.generateFromText(desc);
    UMLDiagram diagram = model.toUMLDiagram();
    diagram.setProjectId(projectId);
    diagram.setSourceType(SourceType.NATURAL_LANGUAGE);
    diagramRepo.save(diagram);
    return diagram;
}
```

A.2 DiagramController — analyzeUploadedImage

```
public UMLDiagram analyzeUploadedImage(UUID projectId, byte[] imageData) {
    if (imageData == null || imageData.length < 4) {
        throw new UnsupportedFileException("Invalid image payload.");
    }
```

```
    boolean isPng = imageData[0]==(byte)0x89 && imageData[1]==(byte)0x50
                 && imageData[2]==(byte)0x4E && imageData[3]==(byte)0x47;
    boolean isJpg = imageData[0]==(byte)0xFF && imageData[1]==(byte)0xD8
                 && imageData[2]==(byte)0xFF;
    if (!isPng && !isJpg) {
        throw new UnsupportedFileException("Only PNG/JPG files are accepted.");
    }
    UMLModel model = aiEngine.analyzeImage(imageData);
    // ...
}
```

A.3 AIController — evaluateDesign

```
public DesignEvaluationReport evaluateDesign(UUID diagramId) {
    UMLDiagram diagram = diagramRepo.findById(diagramId);
    if (diagram == null || diagram.getClasses().size() < 2) {
        throw new DiagramTooSimpleException("Need at least 2 classes.");
    }
    UMLModel model = diagram.toUMLModel();
    DesignEvaluationReport report = aiEngine.evaluateDesign(model);
    report.setProjectId(diagram.getProjectId());
    evalRepo.save(report);
    return report;
}
```

A.4 SQLite Schema — Core Tables

```
CREATE TABLE IF NOT EXISTS project (
    project_id TEXT PRIMARY KEY,
    name TEXT NOT NULL UNIQUE,
    description TEXT,
    created_date TEXT,
    last_modified TEXT
);

CREATE TABLE IF NOT EXISTS uml_diagram (
    diagram_id TEXT PRIMARY KEY,
    project_id TEXT NOT NULL REFERENCES project(project_id) ON DELETE CASCADE,
    source_type TEXT CHECK(source_type IN ('NATURAL_LANGUAGE','SOURCE_CODE','
    MANUAL','UPLOADED_IMAGE')),
    render_state TEXT DEFAULT 'PENDING',
    last_updated TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS conceptual_class (
    class_id TEXT PRIMARY KEY,
    diagram_id TEXT NOT NULL REFERENCES uml_diagram(diagram_id) ON DELETE
    CASCADE,
    class_name TEXT NOT NULL,
    class_type TEXT DEFAULT 'ENTITY',
    position_x REAL DEFAULT 100,
    position_y REAL DEFAULT 100
);
```

A.5 Unit Test Summary

Test Class	Tests	Pass Rate	Key Scenarios
`DiagramControllerTest`	10	100%	Short desc, code parse, null format, blank path, magic bytes, rename
`AIControllerTest`	4	100%	Too simple diagram, valid question, null project ID, empty question
`ProjectControllerTest`	5	100%	Duplicate name, blank name, create valid, open valid, update metadata
`DiagramRepositoryImplTest`	6	100%	Save, findById, findByProject, delete, update, cascade delete
`LLMAPIEngineTest`	4	100%	Valid prompt, API error handling, JSON parse, image encode
`JavaCodeParserTest`	5	100%	Parse single class, parse with inheritance, interface detection, empty file
`DiagramExportServiceTest`	3	100%	PNG export, SVG export, PDF export
`DatabaseManagerTest`	2	100%	Singleton instance, schema initialisation
`RelationshipEdgeTest`	3	100%	Association render, Inheritance arrow, Dependency dashed
`JavaSyntaxHighlighterTest`	2	100%	Keyword colour, string literal colour